

7.二叉树结构



二叉树结构基础

二叉树基础及常见类型

几种常见的二叉树

满二叉树

直接看图比较直观，满二叉树就是每一层节点都是满的，整棵树像一个正三角形：

满二叉树有个优势，就是它的节点个数很好算。假设深度为 h ，那么总节点数就是 $2^h - 1$ ，等比数列求和。

完全二叉树

完全二叉树是指，二叉树的每一层的节点都紧凑靠左排列，且除了最后一层，其他每层都必须满足的：

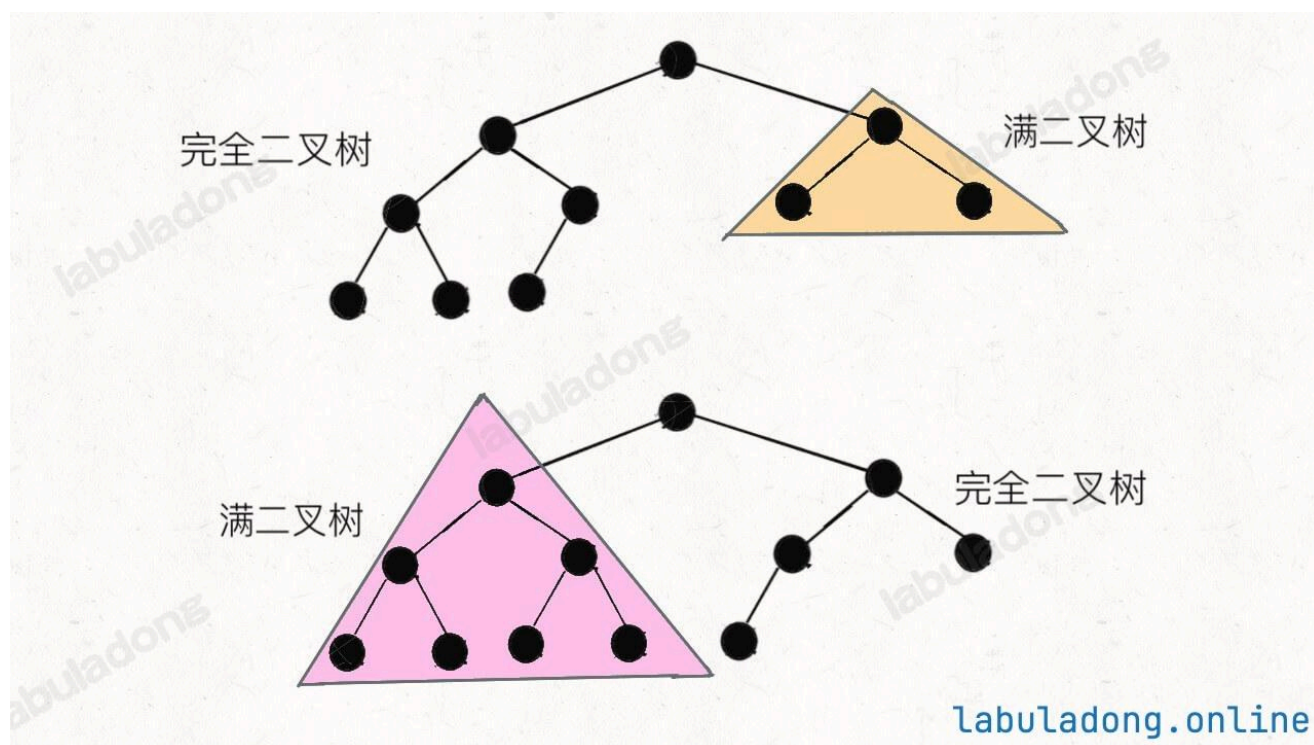
不难发现，满二叉树其实是一种特殊的完全二叉树。

完全二叉树的特点：由于它的节点紧凑排列，如果从左到右从上到下对它的每个节点编号，那么父子节点的索引存在明显的规律。

这个特点在讲到 [二叉堆核心原理](#) 和 [线段树核心原理](#) 时会用到：完全二叉树可以用数组来存储，不需要真的构建链式节点。

完全二叉树还有个比较难发觉的性质：**完全二叉树的左右子树也是完全二叉树。**

或者更准确地说应该是：**完全二叉树的左右子树中，至少有一棵是满二叉树。**



这个性质在做算法题的时候会用到，比如 [巧算完全二叉树的节点数](#)，这里就先提一下。

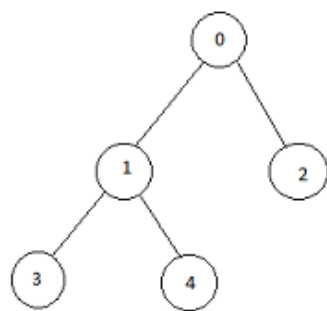
中英文的定义有区别

关于完全二叉树和满二叉树的定义，中文语境和英文语境似乎有点区别。

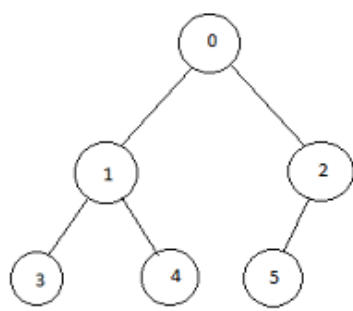
我们说的完全二叉树对应英文 Complete Binary Tree，这个没问题，说的是同一种树。

我们说的满二叉树，按理说应该翻译成 Full Binary Tree 对吧，但其实不是，满二叉树的定义对应英文的 Perfect Binary Tree。

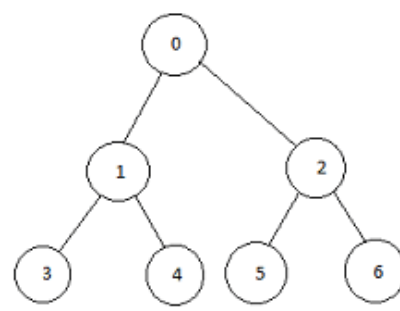
而英文中的 Full Binary Tree 是指一棵二叉树的所有节点要么没有孩子节点，要么有两个孩子节点。



Full
Binary Tree



Complete
Binary Tree



Perfect
Binary Tree

以上定义出自 wikipedia，这里就是顺便一提。其实名词叫什么都无所谓，你知道有这个区别，在看英文资料时留意一下就行了。

二叉搜索树

二叉搜索树（Binary Search Tree，简称 BST）是一种很常见的二叉树，它的定义是：

对于树中的每个节点，其**左子树的每个节点**的值都要小于这个节点的值，**右子树的每个节点**的值都要大于这个节点的值。你可以简单记为「左小右大」。

BST 是非常常用的数据结构。因为左小右大的特性，可以让我们在 BST 中快速找到某个节点，或者找到某个范围内的所有节点，这是 BST 的优势所在。

比方说，对于一棵普通的二叉树，其中的节点大小没有任何规律可言，那么你要找到某个值为 x 的节点，只能从根节点开始遍历整棵树。

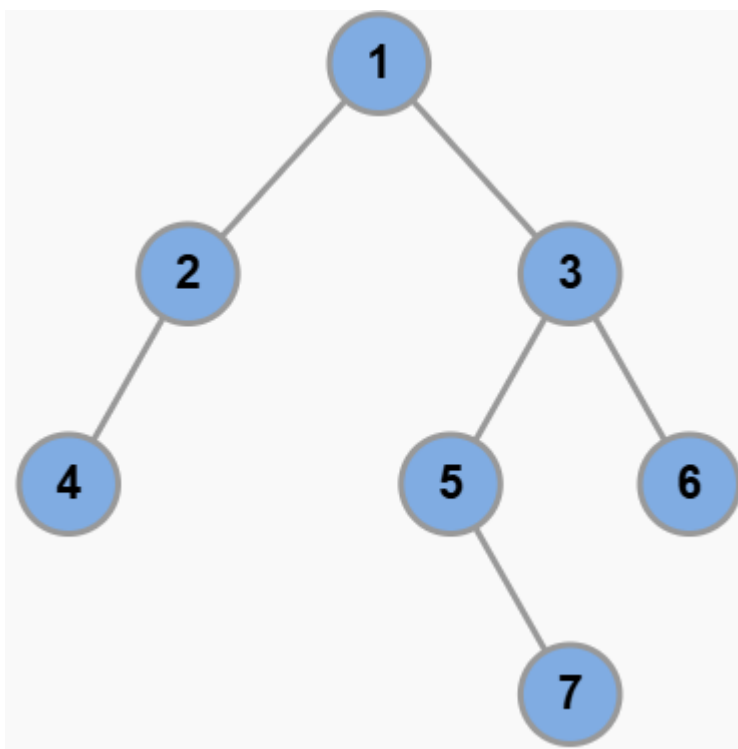
而对于 BST，你可以先对比根节点和 x 的大小关系，如果 x 比根节点大，那么根节点的整棵左子树就可以直接排除了，直接从右子树开始找，这样就可以快速定位到值为 x 的那个节点。

高度平衡二叉树

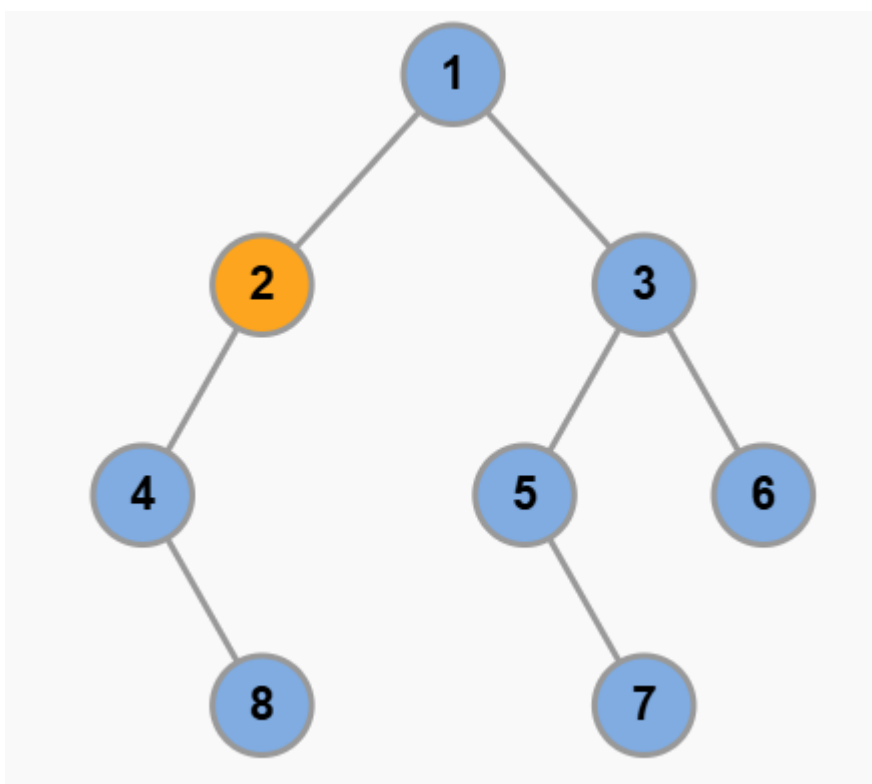
高度平衡二叉树（Height-Balanced Binary Tree）是一种特殊的二叉树，它的「每个节点」的左右子树的高度差不超过 1。

要注意是每个节点，而不仅仅是根节点。

比如下面这棵二叉树，根节点 1 的左子树高度是 2，右子树高度是 3；节点 2 的左子树高度是 1，右子树高度是 0；节点 3 的左子树高度是 2，右子树高度是 1，以此类推，每个节点的左右子树高度差都不超过 1，所以这是一棵高度平衡的二叉树：



下面这棵树就不是高度平衡的二叉树，因为节点 2 的左子树高度是 2，右子树高度是 0，高度差超过 1，不符合条件：



假设高度平衡二叉树中共有 N 个节点，那么高度平衡二叉树的高度是 $O(\log N)$ 。这是非常重要的性质，本站后面的章节会讲解几种基于二叉树的数据结构，如果能保证树的高度为 $O(\log N)$ ，那么这些数据结构的增删查改效率就会很高。

反之，如果树很不平衡，那么这棵树其实就等同于单链表，在树中进行增删查改的效率就会大幅降低。

自平衡二叉树

上面介绍了高度平衡二叉树，说到它的高度为 $O(\log N)$ ，增删查改的效率高。

如果我們可以在增删二叉树节点时对树的结构进行一些调整，那么就可以让树的高度始终是平衡的，这就是自平衡二叉树（Self-Balanced Binary Tree）。

自平衡的二叉树有很多种实现方式，最经典的就是 [红黑树](#)，一种自平衡的二叉搜索树。

二叉树的实现

最常见的二叉树就是类似链表那样的链式存储结构，每个二叉树节点有指向左右子节点的指针，这种方式比较简单直观。

力扣/LeetCode 上给你输入的二叉树一般都是用这种方式构建的，二叉树节点类 `TreeNode` 一般长这样：

```
class TreeNode {
public:
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

// 你可以这样构建一棵二叉树：

```
TreeNode* root = new TreeNode(1);
root->left = new TreeNode(2);
root->right = new TreeNode(3);
root->left->left = new TreeNode(4);
root->right->left = new TreeNode(5);
root->right->right = new TreeNode(6);
```

// 构建出来的二叉树是这样的：

```
//      1
//     / \
//    2   3
//   / \ / \
//  4  5 6
```

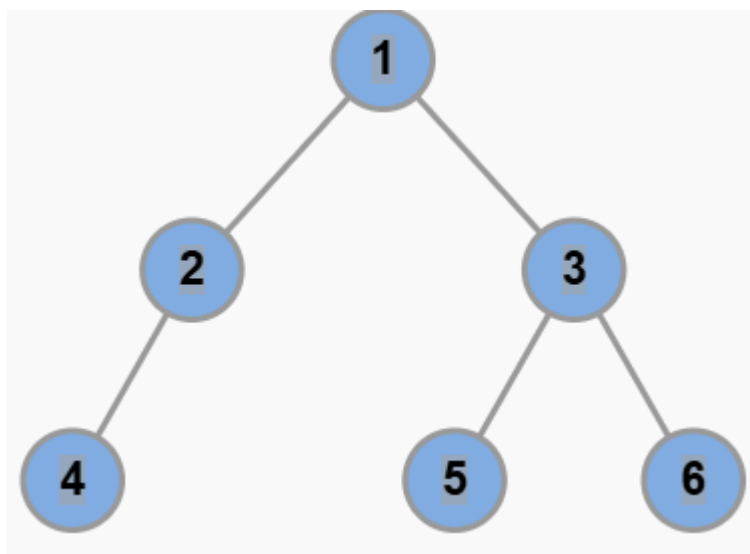
既然说上面是比较常见的实现方式，那言下之意就是还有其他实现方式，对吧？

是的，在 [二叉堆原理及实现](#) 和 [并查集算法详解](#) 中，我们会根据具体的需求场景选择用数组来存储二叉树。

在 [可视化面板](#) 可视化递归函数时，其实是根据函数堆栈生成的递归树，这也可以算是一种二叉树的实现方式。

另外，在一般的算法题中，我们可能会把实际问题**抽象**成二叉树结构，但我们并不需要真的用 `TreeNode` 创建一棵二叉树出来，而是直接用类似 [哈希表](#) 的结构来表示二叉树/多叉树。

我可以用一个哈希表，其中的键是父节点 id，值是子节点 id 的列表（每个节点的 id 是唯一的），那么一个键值对就是一个多叉树节点了，这棵多叉树就可以表示成这样：



```
// 1 -> {2, 3}
// 2 -> {4}
// 3 -> {5, 6}

unordered_map<int, vector<int>> tree;
tree[1] = {2, 3};
tree[2] = {4};
tree[3] = {5, 6};
```

这样就可以模拟和操作二叉树/多叉树结构，后文讲到图论的时候你就会知道，它有一个新的名字叫做 [邻接表](#)。

二叉树的递归/层序遍历

一句话总结：

二叉树只有**递归遍历**和**层序遍历**这两种，再无其他。递归遍历可以衍生出 DFS 算法，层序遍历可以衍生出 BFS 算法。

递归遍历二叉树节点的顺序是固定的，但是有三个关键位置，在不同位置插入代码，会产生不同的效果。

层序遍历二叉树节点的顺序也是固定的，但是有三种不同的写法，对应不同的场景。

递归遍历（DFS）

递归遍历二叉树的代码模板如下：

```
// 基本的二叉树节点
class TreeNode {
public:
    int val;
    TreeNode* left;
    TreeNode* right;
```

```

    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

// 二叉树的递归遍历框架
void traverse(TreeNode* root) {
    if (root == nullptr) {
        return;
    }
    traverse(root->left);
    traverse(root->right);
}

```

请问，这段短小精干的代码为什么能遍历二叉树？又是以什么顺序遍历二叉树的？

对于 `traverse` 这样的递归遍历函数，你就可以把它理解成一个在二叉树结构上游走的指针，

`traverse` 函数的遍历顺序就是一直往左子节点走，直到遇到空指针不能再走了，才尝试往右子节点走一步；然后再一直尝试往左子节点走，如此循环；如果左右子树都走完了，则返回上一层父节点。

看代码也能看出来，先递归调用的 `root.left`，然后才递归调用的 `root.right`，每次进入 `traverse` 函数，都会先往左子节点递归遍历，直到遇到空指针走不动了，才轮到往右子节点走一次。

那么我们简单拓展一下，如果修改前面的 `traverse` 函数，先递归遍历 `root.right`，再递归遍历 `root.left`，会是什么效果？

```

// 修改标准的二叉树遍历框架
void traverseFlip(TreeNode* root) {
    if (root == nullptr) {
        return;
    }
    // 反过来，先递归遍历右子树，再递归遍历左子树
    traverseFlip(root->right);
    traverseFlip(root->left);
}

```

可以看到 `traverseFlip` 函数也能遍历二叉树的所有节点，只不过和标准的 `traverse` 函数遍历顺序相反。

我举这个 `traverseFlip` 的例子，是想告诉你：

递归遍历节点的顺序仅取决于左右子节点的递归调用顺序，与其他代码无关。

我们说二叉树遍历时，一般不会像 `traverseFlip` 这样遍历二叉树，默认还是按照先左后右的顺序，所以当我们说二叉树遍历的代码模板时，指的是先左后右的遍历顺序：

只要这个先左后右的调用顺序不变，那么 `traverse` 函数访问节点的顺序就是固定的，你插入一万行代码进去，也不会变。

理解前/中/后序遍历

递归遍历的顺序，即 `traverse` 函数访问节点的顺序确实是固定的。

但是，你在 `traverse` 函数中不同位置写代码，效果是可以不一样的。前中后序遍历的结果不同，原因是因为你把代码写在了不同位置，所以产生了不同的效果。

比方说，刚进入一个节点的时候，你还对它的子节点一无所知，而当你离开一个节点的时候，它的所有子节点你都遍历过了。那么在这两种情况下写的代码，肯定是可以有不同的效果的。

所谓的前中后序遍历，其实就是在二叉树遍历框架的不同位置写代码：

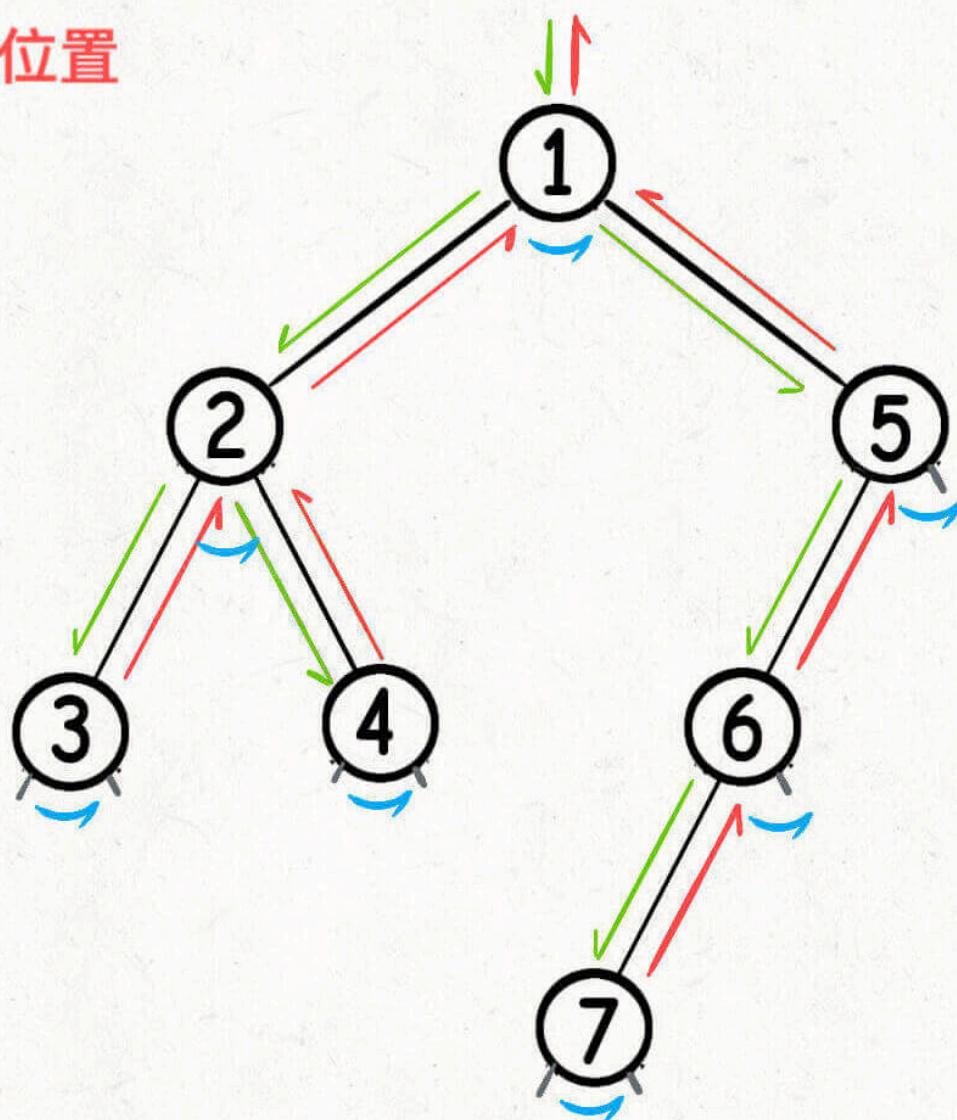
```
// 二叉树的遍历框架
void traverse(TreeNode* root) {
    if (root == nullptr) {
        return;
    }
    // 前序位置
    traverse(root->left);
    // 中序位置
    traverse(root->right);
    // 后序位置
}
```

前序位置的代码会在进入节点时立即执行；中序位置的代码会在左子树遍历完成后，遍历右子树之前执行；后序位置的代码会在左右子树遍历完成后执行：

前序位置

中序位置

后序位置

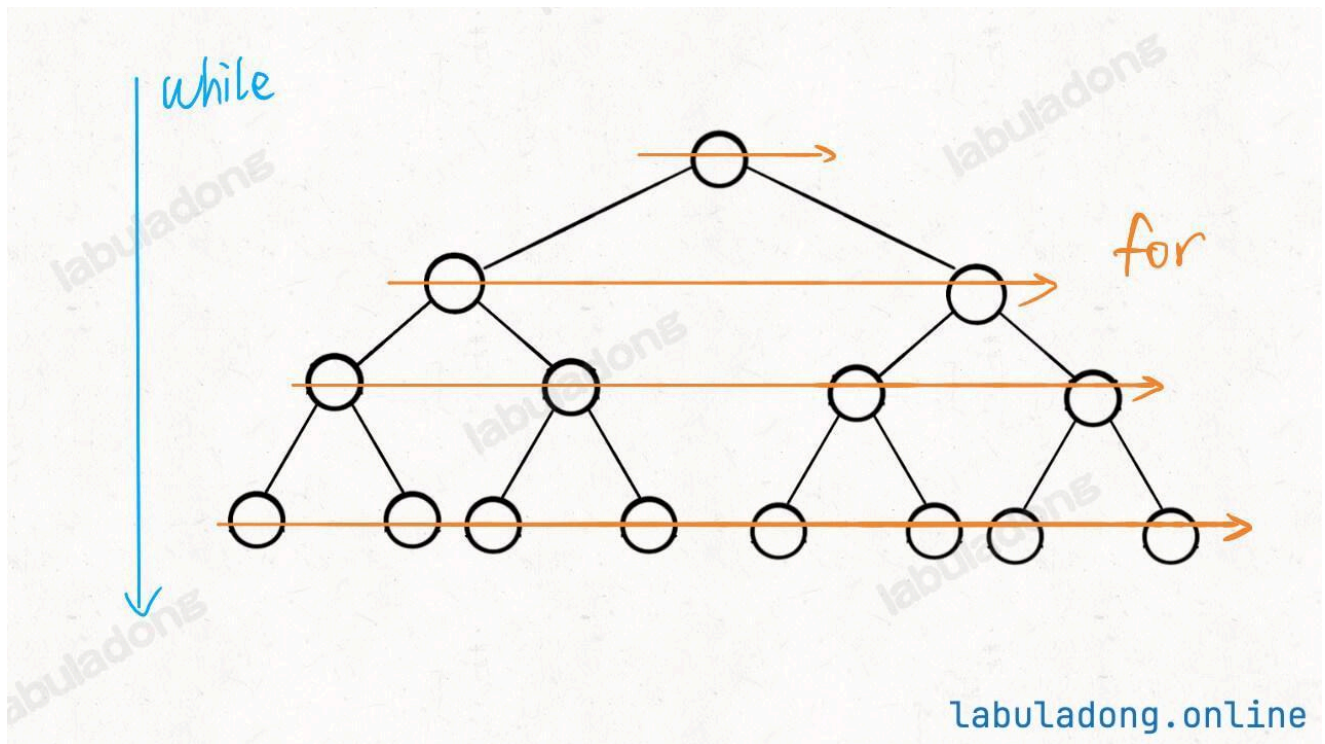


公众号: labuladong

层序遍历 (BFS)

上面讲的递归遍历是依赖函数堆栈递归遍历二叉树的，遍历顺序是从最左侧开始，一系列地走到最右侧。

二叉树的层序遍历，顾名思义，就是一层一层地遍历二叉树：



层序遍历需要借助队列来实现，而且根据不同的需求，可以有三种不同的写法，下面一一列举。

写法一

```
void levelOrderTraverse(TreeNode* root) {
    if (root == nullptr) {
        return;
    }
    std::queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        TreeNode* cur = q.front();
        q.pop();
        // 访问 cur 节点
        std::cout << cur->val << std::endl;

        // 把 cur 的左右子节点加入队列
        if (cur->left != nullptr) {
            q.push(cur->left);
        }
        if (cur->right != nullptr) {
            q.push(cur->right);
        }
    }
}
```

这种写法最大的优势就是简单。每次把队头元素拿出来，然后把它的左右子节点加入队列，就完事了。

但是这种写法的缺点是，无法知道当前节点在第几层。知道节点的层数是个常见的需求，比方说让你收集每一层的节点，或者计算二叉树的最小深度等等。

所以这种写法虽然简单，但用的不多，下面介绍的写法会更常见一些。

写法二

```
void levelOrderTraverse(TreeNode* root) {
    if (root == nullptr) {
        return;
    }
    queue<TreeNode*> q;
    q.push(root);
    // 记录当前遍历到的层数（根节点视为第 1 层）
    int depth = 1;

    while (!q.empty()) {
        int sz = q.size();
        for (int i = 0; i < sz; i++) {
            TreeNode* cur = q.front();
            q.pop();
            // 访问 cur 节点，同时知道它所在的层数
            cout << "depth = " << depth << ", val = " << cur->val << endl;

            // 把 cur 的左右子节点加入队列
            if (cur->left != nullptr) {
                q.push(cur->left);
            }
            if (cur->right != nullptr) {
                q.push(cur->right);
            }
        }
        depth++;
    }
}
```

注意代码中的内层 for 循环：

```
int sz = q.size();
for (int i = 0; i < sz; i++) {
    ...
}
```

这个变量 `i` 记录的是节点 `cur` 是当前层的第几个，大部分算法题中都不会用到这个变量，所以你完全可以改用下面的写法：

```
int sz = q.size();
while (sz-- > 0) {
    ...
}
```

这个属于细节问题，按照自己的喜好来就行。

但是注意队列的长度 `sz` 一定要在循环开始前保存下来，因为在循环过程中队列的长度是会变化的，不能直接用 `q.size()` 作为循环条件。

这种写法就可以记录下来每个节点所在的层数，可以解决诸如二叉树最小深度这样的问题，是我们最常用的层序遍历写法。

写法三

既然写法二是最常见的，为啥还有个写法三呢？因为要给后面的进阶内容做铺垫。

现在我们只是在探讨二叉树的层序遍历，但是二叉树的层序遍历可以衍生出 [多叉树的层序遍历](#)，[图的 BFS 遍历](#)，以及经典的 [BFS 暴力穷举算法框架](#)，所以这里要拓展延伸一下。

回顾写法二，我们每向下遍历一层，就给 `depth` 加 1，可以理解为每条树枝的权重是 1，二叉树中每个节点的深度，其实就是从根节点到这个节点的路径权重和，且同一层的所有节点，路径权重和都是相同的。

那么假设，如果每条树枝的权重和可以是任意值，现在让你层序遍历整棵树，打印每个节点的路径权重和，你会怎么做？

这样的话，同一层节点的路径权重和就不一定相同了，写法二这样只维护一个 `depth` 变量就无法满足需求了。

写法三就是为了解决这个问题，在写法一的基础上添加一个 `State` 类，让每个节点自己负责维护自己的路径权重和，代码如下：

```
class State {
public:
    TreeNode* node;
    int depth;

    State(TreeNode* node, int depth) : node(node), depth(depth) {}
};

void levelOrderTraverse(TreeNode* root) {
    if (root == nullptr) {
        return;
    }
    queue<State> q;
    // 根节点的路径权重和是 1
    q.push(State(root, 1));
```

```

while (!q.empty()) {
    State cur = q.front();
    q.pop();
    // 访问 cur 节点，同时知道它的路径权重和
    cout << "depth = " << cur.depth << ", val = " << cur.node->val <<
endl;

    // 把 cur 的左右子节点加入队列
    if (cur.node->left != nullptr) {
        q.push(State(cur.node->left, cur.depth + 1));
    }
    if (cur.node->right != nullptr) {
        q.push(State(cur.node->right, cur.depth + 1));
    }
}
}

```

这样每个节点都有了自己的 `depth` 变量，是最灵活的，可以满足所有 **BFS** 算法的需求。但是由于要额外定义一个 `State` 类比较麻烦，所以非必要的话，用写法二就够了。

其实你很快就会学到，这种边带有权重的场景属于图结构算法，在之后的 [BFS 算法习题集](#) 和 [dijkstra 算法](#) 中，才会用到这种写法。

第二个写法需要 `sz` 主要是为了一层层扩散，从而维护 `depth`；第三个写法每个 `State` 自己可以维护和起点之间的距离，维护这个 `sz` 一层一层往外扩散意义是什么呢？

况且写法三针对的场景中，每条边的权重可以不一样，所以这个「层」的概念已经不重要了。

其他遍历？

二叉树的遍历方式只有上面两种，也许有其他的写法，但都是表现形式上的差异，本质上不可能跳出上面两种遍历方式。

比方说，你可能看到用栈来迭代遍历二叉树的代码。但这本质还是递归遍历，只不过他手动维护栈模拟递归调用罢了。

再比如，你还可能看到递归地一层层遍历二叉树的代码。但这本质还是层序遍历，只不过他把层序遍历代码中的 `for` 循环用递归的形式展现了。

总之，不要被表象迷惑，二叉树的遍历方式就上面两种，结合后面的教程和习题，你把这两种遍历方式玩明白，一切暴力穷举算法都小菜一碟。

DFS 和 BFS 的适用场景

为什么 BFS 常用来寻找最短路径

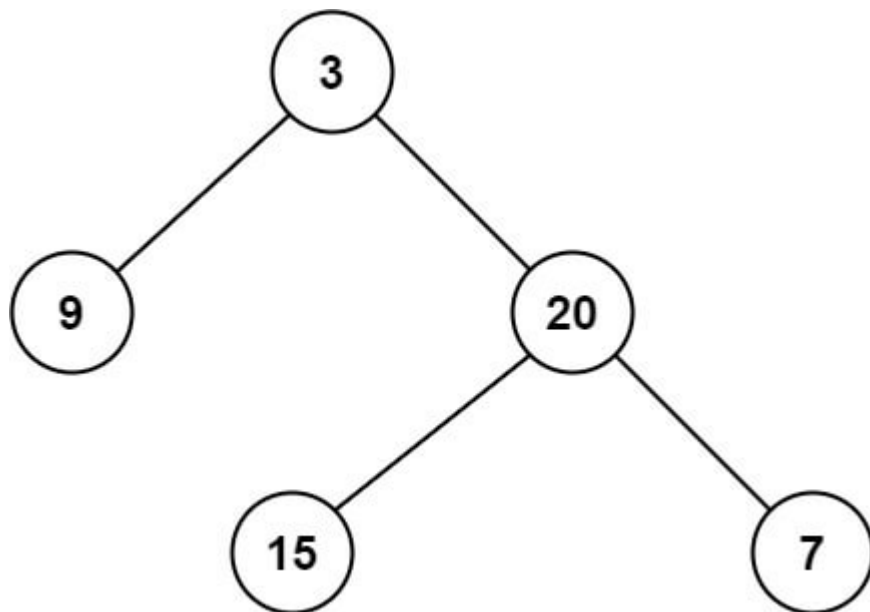
111. 二叉树的最小深度 | [力扣](#) | [LeetCode](#) | ●

给定一个二叉树，找出其最小深度。

最小深度是从根节点到最近叶子节点的最短路径上的节点数量。

说明： 叶子节点是指没有子节点的节点。

示例 1：



输入： root = [3,9,20,null,null,15,7]

输出： 2

示例 2：

输入： root = [2,null,3,null,4,null,5,null,6]

输出： 5

提示：

- 树中节点数的范围在 [0, 10⁵] 内
- $-1000 \leq \text{Node.val} \leq 1000$

二叉树的最小深度即「**根节点到最近的叶子节点的距离**」，所以这道题本质上就是让你求最短距离。

DFS 递归遍历和 BFS 层序遍历都可以解决这道题，先看 DFS 递归遍历的解法：

```
class Solution {
private:
    // 记录最小深度（根节点到最近的叶子节点的距离）
    int minDepthValue = INT_MAX;
    // 记录当前遍历到的节点深度
    int currentDepth = 0;

public:
    int minDepth(TreeNode* root) {
```

```

        if (root == nullptr) {
            return 0;
        }

        // 从根节点开始 DFS 遍历
        traverse(root);
        return minDepthValue;
    }

    void traverse(TreeNode* root) {
        if (root == nullptr) {
            return;
        }

        // 前序位置进入节点时增加当前深度
        currentDepth++;

        // 如果当前节点是叶子节点，更新最小深度
        if (root->left == nullptr && root->right == nullptr) {
            minDepthValue = min(minDepthValue, currentDepth);
        }

        traverse(root->left);
        traverse(root->right);

        // 后序位置离开节点时减少当前深度
        currentDepth--;
    }
};

```

每当遍历到一条树枝的叶子节点，就会更新最小深度，**当遍历完整棵树后**，就能算出整棵树的最小深度。

你能不能在不遍历完整棵树的情况下，提前结束算法？不可以，因为你必须确切的知道每条树枝的深度（根节点到叶子节点的距离），才能找到最小的那个。

下面来看 BFS 层序遍历的解法。按照 BFS 从上到下逐层遍历二叉树的特点，当遍历到第一个叶子节点时，就能得到最小深度：

```

class Solution {
public:
    int minDepth(TreeNode* root) {
        if (root == nullptr) return 0;
        queue<TreeNode*> q;
        q.push(root);
        // root 本身就是一层，depth 初始化为 1
        int depth = 1;

        while (!q.empty()) {

```

```

        int sz = q.size();
        // 遍历当前层的节点
        for (int i = 0; i < sz; i++) {
            TreeNode* cur = q.front();
            q.pop();
            // 判断是否到达叶子结点
            if (cur->left == nullptr && cur->right == nullptr)
                return depth;
            // 将下一层节点加入队列
            if (cur->left != nullptr)
                q.push(cur->left);
            if (cur->right != nullptr)
                q.push(cur->right);
        }
        // 这里增加步数
        depth++;
    }
    return depth;
}
};

```

当它遍历到第二行的时候，就遇到第一个叶子节点了，这个叶子节点就是距离根节点最近的叶子节点，所以此时算法就结束了。BFS 算法并没有遍历整棵树就找到了最小深度。

综上，你应该能理解为啥 BFS 算法经常用来寻找最短路径了：

由于 BFS 逐层遍历的逻辑，第一次遇到目标节点时，所经过的路径就是最短路径，算法可能并不需要遍历完所有节点就能提前结束。

DFS 遍历当然也可以用来寻找最短路径，但必须遍历完所有节点才能得到最短路径。

从时间复杂度的角度来看，两种算法在最坏情况下都会遍历所有节点，时间复杂度都是 $O(N)$ ，但在一般情况下，显然 BFS 算法的实际效率会更高。所以在寻找最短路径的问题中，BFS 算法是首选。

为什么 DFS 常用来寻找所有路径

在寻找所有路径的问题中，你会发现 DFS 算法用的比较多，BFS 算法似乎用的不多。

理论上两种遍历算法都是可以的，只不过 BFS 算法寻找所有路径的代码比较复杂，DFS 算法代码比较简洁。

你想啊，就以二叉树为例，如果要用 BFS 算法来寻找所有路径（根节点到每个叶子节点都是一条路径），队列里面就不能只放节点了，而需要使用 [二叉树层序遍历的第三种写法](#)，新建一个 State 类，把当前节点以及到达当前节点的路径都存进去，这样才能正确维护每个节点的路径，最终计算出所有路径。

而使用 DFS 算法就简单了，它本就是一条树枝一条树枝从左往右遍历的，每条树枝就是一条路径，递归遍历到叶子节点的时候递归路径就是一条树枝，所以 DFS 算法天然适合寻找所有

路径。

综上，DFS 算法在寻找所有路径的问题中更常用，而 BFS 算法在寻找最短路径的问题中更常用。

多叉树的递归/层序遍历

叉树的节点长这样，每个节点有任意个子节点：

```
class Node {
public:
    int val;
    vector<Node*> children;
};
```

就这点区别，其他没了。

森林

这里介绍一下「森林」这个名词，后面讲到 [Union Find 并查集算法](#) 时，会用到这个概念。

顾名思义，**森林就是多个多叉树的集合（单独一棵多叉树也是一个特殊的森林）**，用代码表示就是多个多叉树的根节点列表，类似这样：

```
List<Node> forest;
```

只需对每个根节点分别进行 DFS/BFS 遍历，即可遍历森林的所有节点。

在并查集算法中，我们会同时持有多棵多叉树的根节点，那么这些根节点的集合就是一个森林。

接下来说下多叉树的遍历，和二叉树一样，也就递归遍历（DFS）和层序遍历（BFS）两种。

递归遍历（DFS）

对比二叉树的遍历框架看多叉树的遍历框架吧

```
// 二叉树的遍历框架
void traverse(TreeNode* root) {
    if (root == nullptr) {
        return;
    }
    // 前序位置
    traverse(root->left);
    // 中序位置
    traverse(root->right);
    // 后序位置
}
```

```

// N 叉树的遍历框架
void traverse(Node* root) {
    if (root == nullptr) {
        return;
    }
    // 前序位置
    for (Node* child : root->children) {
        traverse(child);
    }
    // 后序位置
}

```

唯一的区别是，多叉树没有了中序位置，因为可能有多个节点嘛，所谓的中序位置也就没什么意义了。

层序遍历 (BFS)

多叉树的层序遍历和 二叉树的层序遍历 一样，都是用队列来实现，无非就是把二叉树的左右子节点换成了多叉树的所有子节点。所以多叉树的层序遍历也有三种写法，下面一一列举。

```

void levelOrderTraverse(Node* root) {
    if (root == nullptr) {
        return;
    }
    std::queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* cur = q.front();
        q.pop();
        // 访问 cur 节点
        std::cout << cur->val << std::endl;

        // 把 cur 的所有子节点加入队列
        for (Node* child : cur->children) {
            q.push(child);
        }
    }
}

```

```

#include <iostream>
#include <queue>
#include <vector>

void levelOrderTraverse(Node* root) {
    if (root == nullptr) {
        return;
    }
}

```

```

std::queue<Node*> q;
q.push(root);
// 记录当前遍历到的层数（根节点视为第 1 层）
int depth = 1;

while (!q.empty()) {
    int sz = q.size();
    for (int i = 0; i < sz; i++) {
        Node* cur = q.front();
        q.pop();
        // 访问 cur 节点，同时知道它所在的层数
        std::cout << "depth = " << depth << ", val = " << cur->val <<
std::endl;

        for (Node* child : cur->children) {
            q.push(child);
        }
    }
    depth++;
}
}

```

```

// 多叉树的层序遍历
// 每个节点自行维护 State 类，记录深度等信息
class State {
public:
    Node* node;
    int depth;

    State(Node* node, int depth) : node(node), depth(depth) {}
};

void levelOrderTraverse(Node* root) {
    if (root == nullptr) {
        return;
    }
    std::queue<State> q;
    // 记录当前遍历到的层数（根节点视为第 1 层）
    q.push(State(root, 1));

    while (!q.empty()) {
        State state = q.front();
        q.pop();
        Node* cur = state.node;
        int depth = state.depth;
        // 访问 cur 节点，同时知道它所在的层数
        std::cout << "depth = " << depth << ", val = " << cur->val <<
std::endl;

```

```
        for (Node* child : cur->children) {
            q.push(State(child, depth + 1));
        }
    }
}
```

二叉树结构的种种变换基础

二叉搜索树

二叉搜索树是特殊的 二叉树结构，其主要的实际应用是 `TreeMap` 和 `TreeSet`。

二叉搜索树的优势

二叉搜索树（BST）的特点，即左小右大：

对于树中的每个节点，其**左子树的每个节点**的值都要小于这个节点的值，**右子树的每个节点**的值都要大于这个节点的值。

这个左小右大的特性，可以让我们在 BST 中快速找到某个节点，或者找到某个范围内的所有节点，这是 BST 的优势所在。利用 BST 左小右大的特性，可以迅速定位到目标节点，理想的时间复杂度是树的高度 $O(\log N)$ ，而普通的二叉树遍历函数则需要 $O(N)$ 的时间遍历所有节点。

至于其他增、删、改的操作，你首先查到目标节点，才能进行增删改的操作对吧？增删改的操作无非就是改一改指针，所以增删改的时间复杂度也是 $O(\log N)$ 。

TreeMap/TreeSet 实现原理

你看 `TreeMap` 这个名字，应该就能看出来，它和前文介绍的 [哈希表](#) `HashMap` 的结构是类似的，都是存储键值对的，`HashMap` 底层把键值对存储在一个 `table` 数组里面，而 `TreeMap` 底层把键值对存储在一棵二叉搜索树的节点里面。

至于 `TreeSet`，它和 `TreeMap` 的关系正如哈希表 `HashMap` 和哈希集合 `HashSet` 的关系一样，说白了就是 `TreeMap` 的简单封装，所以下面主要讲解 `TreeMap` 的实现原理。

力扣经典的 `TreeNode` 结构长这样：

```
class TreeNode {
public:
    int val;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};
```

我们只要改一改这个经典的 `TreeNode` 结构，就可以用来实现 `TreeMap` 了：

```
// 大写 K 为键的类型，大写 V 为值的类型
template <typename K, typename V>
class TreeNode {
public:
    K key;
    V value;
    TreeNode<K, V>* left;
    TreeNode<K, V>* right;
    TreeNode(K key, V value) : key(key), value(value), left(nullptr),
right(nullptr) {}
};
```

我们将实现的 `TreeMap` 结构有如下 API:

```
// TreeMap 主要接口
class MyTreeMap<K, V> {

    // ***** Map 键值映射的基本方法 *****

    // 增/改，复杂度  $O(\log N)$ 
    public void put(K key, V value) {}

    // 查，复杂度  $O(\log N)$ 
    public V get(K key) {}

    // 删，复杂度  $O(\log N)$ 
    public void remove(K key) {}

    // 是否包含键 key，复杂度  $O(\log N)$ 
    public boolean containsKey(K key) {}

    // 返回所有键的集合，结果有序，复杂度  $O(N)$ 
    public List<K> keys() {}

    // ***** TreeMap 提供的额外方法 *****

    // 查找最小键，复杂度  $O(\log N)$ 
    public K firstKey() {}

    // 查找最大键，复杂度  $O(\log N)$ 
    public K lastKey() {}

    // 查找小于等于 key 的最大键，复杂度  $O(\log N)$ 
    public K floorKey(K key) {}

    // 查找大于等于 key 的最小键，复杂度  $O(\log N)$ 
    public K ceilingKey(K key) {}
```

```
// 查找排名为 k 的键，复杂度 O(logN)
public K selectKey(int k) {}

// 查找键 key 的排名，复杂度 O(logN)
public int rank(K key) {}

// 区间查找，复杂度 O(logN + M)，M 为区间大小
public List<K> rangeKeys(K low, K high) {}
}
```

除了标准的增删查改方法 `get`, `put`, `remove`, `containsKey` 之外，`TreeMap` 还提供了很多额外方法，主要和 `key` 的大小相关。怎么样，是不是感觉很强大？

哈希表很实用，但是它确实没办法很好地处理键之间的大小关系。前文 [用链表加强哈希表](#) 中实现的 `LinkedHashMap` 也只是做到按「插入顺序」排列哈希表中的键，依然做不到按「大小顺序」排列。

基本增删查改

下面简单介绍一下 `TreeMap` 的每个方法的实现原理，具体的代码在后文 [TreeMap/TreeSet 的实现](#) 细说。

首先，`get` 方法根据目标 `key` 和当前节点的 `key` 比较，决定往左走还是往右走，可以一次性排除掉一半的节点，复杂度是 $O(\log N)$ 。

至于 `put`, `remove`, `containsKey` 方法，其实也是要先利用 `get` 方法找到目标键所在的节点，然后做一些指针操作，复杂度都是 $O(\log N)$ 。

`floorKey`, `ceilingKey` 方法是查找小于等于/大于等于某个键的最大/最小键，这个方法的实现和 `get` 方法类似，唯一的区别在于，当找不到目标 `key` 时，`get` 方法返回空指针，而 `ceilingKey`, `floorKey` 方法则返回和目标 `key` 最接近的键，类似上界和下界。

`rangeKeys` 方法输入一个 `[low, hi]` 区间，返回这个区间内的所有键。这个方法的实现也是利用 BST 的性质提高搜索效率：

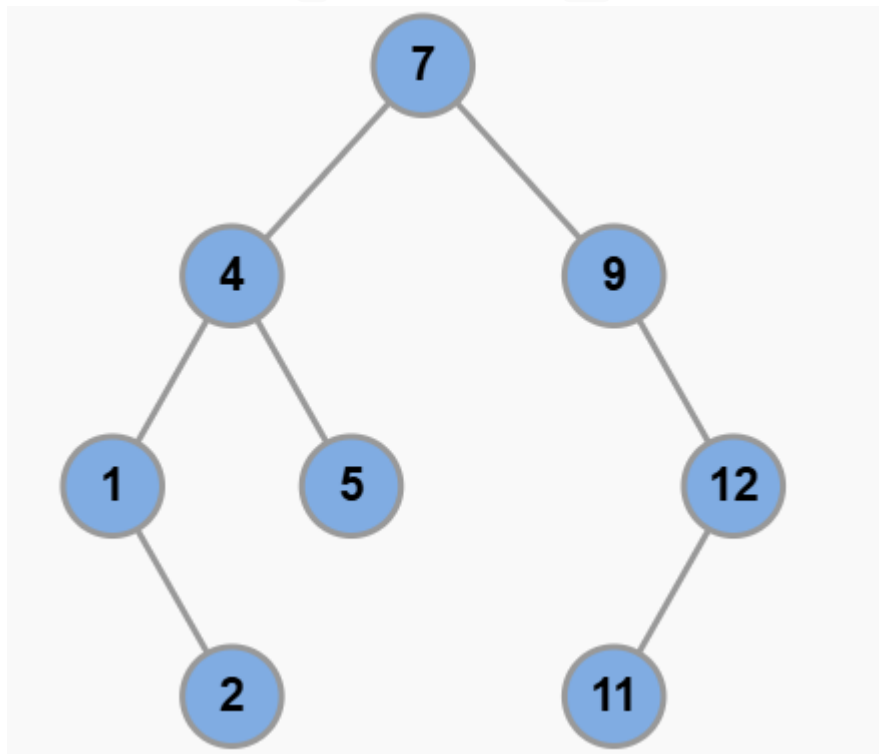
如果当前节点的 `key` 小于 `low`，那么当前节点的整棵左子树都小于 `low`，根本不用搜索了；如果当前节点的 `key` 大于 `hi`，那么当前节点的整棵右子树都大于 `hi`，也不用搜索了。

`firstKey`, `lastKey`, `keys` 方法方法

`firstKey` 方法是查找最小键，`lastKey` 方法是查找最大键，借助 BST 左小右大的特性，非常容易实现：

从 BST 的根节点开始，一直往左走，最后遇到的非空节点就是最小的节点；同理，一直往右走，最后遇到的非空节点就是最大的节点。

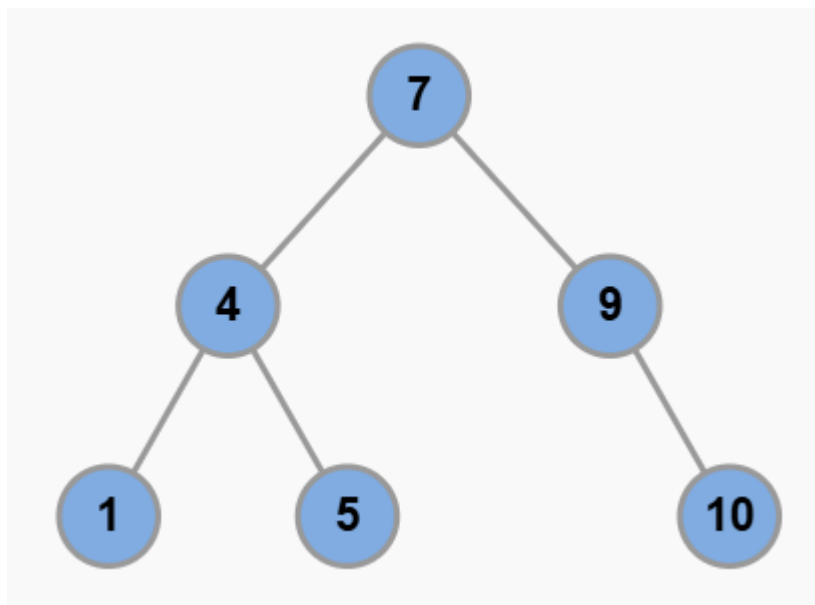
你看看是不是这样，1 是最小的节点，12 是最大的节点：



`selectKey`, `rank` 方法

这两个方法比较有意思。其中 `selectKey` 方法是查找排名为 `k` 的键（从小到大，从 1 开始），`rank` 方法是查找目标 `key` 的排名。

比如下面这棵 BST，`selectKey(3)` 返回 5，因为元素 5 的大小排在第三；`rank(10)` 返回 6，因为元素 10 的排名是第六。



```
keys [1, 4, 5, 7, 9, 10]
rank 1 2 3 4 5 6
```

先说 `selectKey` 方法，一个容易想到的方法是利用 BST 中序遍历结果有序的特性，中序遍历的过程中记录第 `k` 个遍历到节点，就是排名为 `k` 的节点。

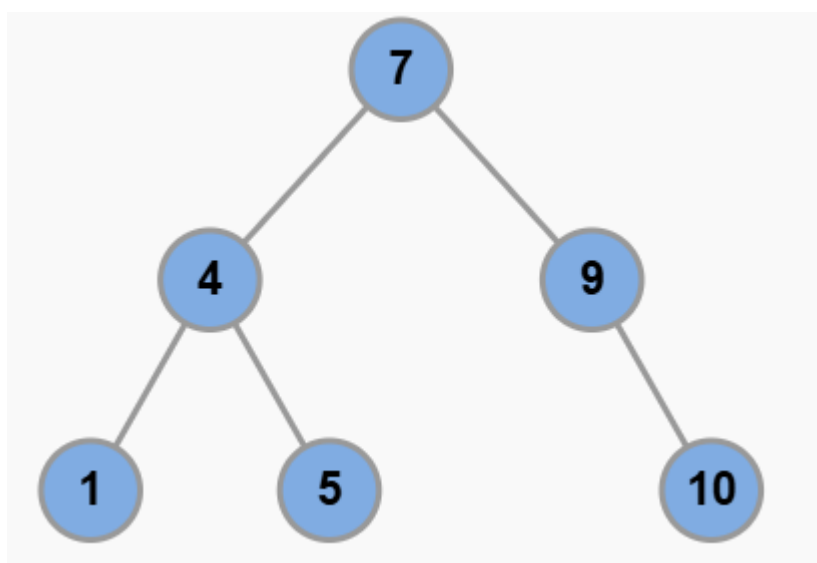
但是这样的时间复杂度是 $O(k)$ ，因为你至少要用中序遍历经过 k 个节点。

一个更巧妙的办法是给二叉树节点中新增更多的字段记录额外信息：

```
class TreeNode<K, V> {
    K key;
    V value;
    // 记录以当前节点为根的子树的节点总数（包含当前节点）
    int size;

    TreeNode<K, V> left;
    TreeNode<K, V> right;

    TreeNode(K key, V value) {
        this.key = key;
        this.value = value;
        this.size = 1;
    }
}
```



```
keys [1, 4, 5, 7, 9, 10]
rank  1  2  3  4  5  6
size  1  3  1  6  2  1
```

这样一来，`selectKey` 就可以利用这个 `size` 字段把时间复杂度降到 $O(\log N)$ 了。

因为 `size` 维护的就是当前节点为根的整棵树上有多少个节点，加上 BST 左小右大的特性，那么对于根节点，它只需要查询左子树的节点个数，就知道了自己在以自己为根的这棵树上的排名；知道了自己的排名，就可以确定目标排名的那个节点存在于左子树还是右子树上，从而避免搜索整棵树。

比方说上面的例子，你调用 `selectKey(3)` 找排名第三的节点，根节点 7 查看左子节点 4 得知左子树共有 3 个节点，加上自己共有 4 个节点，那么 7 就知道自己排名第四。

由此可知，排名第三的节点在 7 的左子树，就不用再往右子树找了。类似的，如果你调用 `selectKey(5)` 找排名第五的节点，7 就知道要往右子树找。

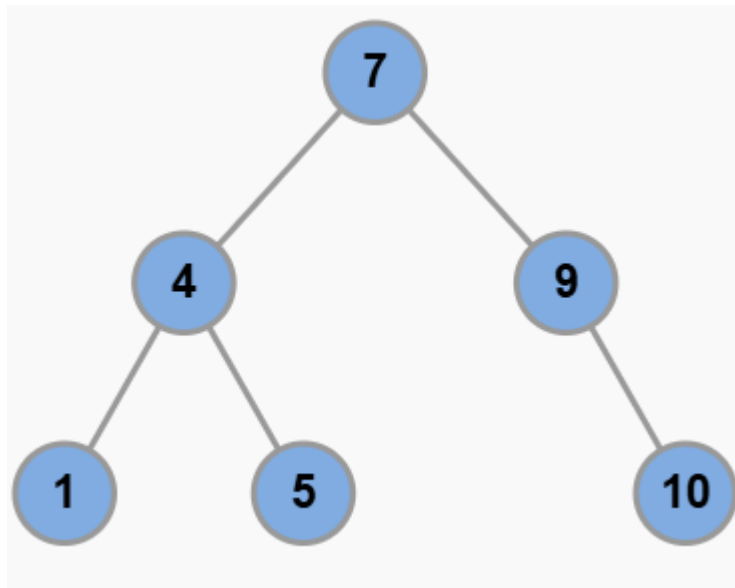
再说 `rank` 方法，也可以利用这个 `size` 字段。比方说你调用 `rank(9)`，想知道节点 9 这个元素排名第几，根节点 7 知道左子树有 3 个节点，加上自己共有 4 个节点，即自己排名第 4；然后 7 可以去右子树递归调用 `rank(9)`，计算节点 9 在右子树中的排名，最后再加上 4，就是整棵树中节点 9 的排名：

```
rank(node(7), 9) = 3 + 1 + rank(node(9), 9) = 3 + 1 + 1 = 5
```

当然，这样也提高了维护的复杂性，因为每次插入、删除节点都要动态地更新这个额外的 `size` 字段，确保它的正确性。

性能问题

前文说复杂度是树的高度 $O(\log N)$ （ N 为节点总数），这是有前提的，即二叉搜索树要是「平衡」的，也就是左右子树的高度差不能太大，比如这样：



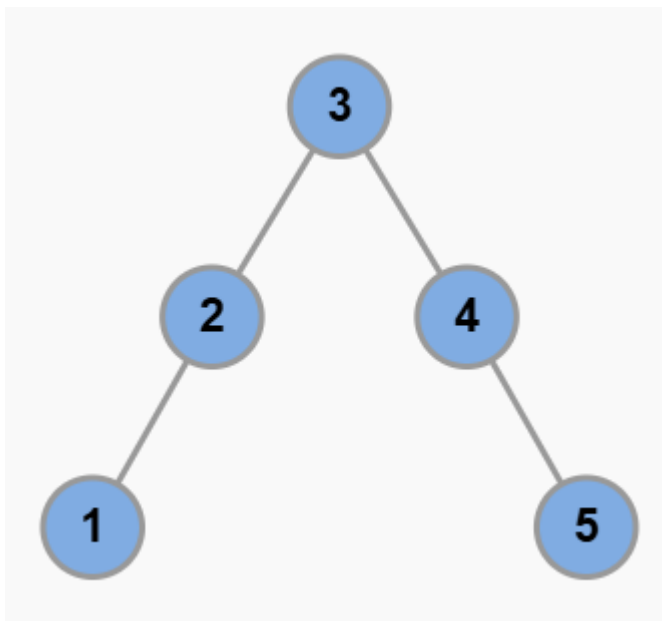
可以直观地感觉到这棵树比较平衡，假设树中节点总数为 N ，那么树的高度是 $O(\log N)$ 。

如果搜索树不平衡，所有节点都只有右子树，没有左子树：

这样二叉搜索树其实退化成了一条单链表，树的高度等于节点数 $O(N)$ ，这种情况下，即便这棵树符合 BST 的定义，但是性能就退化成了链表的性能，增删查改的复杂度全部变成了 $O(N)$ 。

可以这样说：**二叉搜索树的性能取决于树的高度，树的高度取决于树的平衡性**。因此，在实际应用中，`TreeMap` 需要自动维护树的平衡，避免出现性能退化。

如何自动维护平衡性？这可就复杂了。你需要在插入和删除二叉树节点的时候对树进行局部旋转操作，比方上面那棵退化成单链表的 BST，可以这样被旋转成平衡二叉树：



大家熟知的红黑树就是一类自平衡的二叉搜索树，它的平衡性能非常好，但是实现起来比较复杂，这就是完美所需付出的代价。

好了，本文就介绍这么多，上面只说了每个方法的实现思路，具体的代码实现在后文 [TreeMap/TreeSet 的实现](#) 细说

红黑树

红黑树是自平衡的二叉搜索树，它的树高在任何时候都能保持在 $O(\log N)$ （完美平衡），这样就能保证增删查改的时间复杂度都是 $O(\log N)$ 。

Trie/字典树/前缀树

Trie 树就是 [多叉树结构](#) 的延伸，是一种针对字符串进行特殊优化的数据结构。

Trie 树在处理字符串相关操作时有诸多优势，比如节省公共字符串前缀的内存空间、方便处理前缀操作、支持通配符匹配等。

引言

本站将会带你实现一个 `TrieMap` 和 `TrieSet`，先来梳理一下我们已经实现过的 `Map/Set` 类型：

- 标准的 [哈希表](#) `HashMap`，底层借助一个哈希函数把键值对存在 `table` 数组中，有两种解决哈希冲突的方法。它的特点是快，即基本的增删查改操作时间复杂度都是 $O(1)$ 。[哈希集合](#) `HashSet` 是 `HashMap` 的简单封装。
- [哈希链表](#) `LinkedHashMap`，是 [双链表结构](#) 对标准哈希表的加强。它继承了哈希表的操作复杂度，并且可以让哈希表中的所有键保持「插入顺序」。
`LinkedHashSet` 是 `LinkedHashMap` 的简单封装。
- [哈希数组](#) `ArrayHashMap`，是 [数组结构](#) 对标准哈希表的加强。它继承了哈希表的操作复杂度，并且提供了一个额外的 `randomKey` 函数，可以在 $O(1)$ 的时间返回一个随机键。
`ArrayHashSet` 是 `ArrayHashMap` 的简单封装。

- [TreeMap 映射](#)，底层是一棵二叉搜索树（编程语言标准库一般使用经过改良的自平衡 [红黑树](#)），基本增删查改操作复杂度是 $O(\log N)$ ，它的特点是可以动态维护键值对的大小关系，有很多额外的 API 操作键值对。TreeSet 集合是 TreeMap 映射的简单封装。

TrieSet 也是 TrieMap 的简单封装，所以下面我们聚焦 TrieMap 的实现原理即可。

Trie 树的主要应用场景

Trie 树是一种针对字符串有特殊优化的数据结构，这也许它又被叫做字典树的原因。Trie 树针对字符串的处理有若干优势，下面一一列举。

节约存储空间

用 HashMap 对比吧，比如说这样存储几个键值对：

```
Map<String, Integer> map = new HashMap<>();
map.put("apple", 1);
map.put("app", 2);
map.put("appl", 3);
```

回想哈希表的实现原理，键值对会被存到 table 数组中，也就是说它真的创建 "apple"、"app"、"appl" 这三个字符串，占用了 12 个字符的内存空间。

但是注意，这三个字符串拥有共同的前缀，"app" 这个前缀被重复存储了三次，"l" 也被重复存储了两次。

如果换成 TrieMap 来存储：

```
// Trie 树的键类型固定为 String 类型，值类型可以是泛型
TrieMap<Integer> map = new TrieMap<>();
map.put("apple", 1);
map.put("app", 2);
map.put("appl", 3);
```

Trie 树底层并不会重复存储公共前缀，所以只需要 "apple" 这 5 个字符的内存空间来存储键。

这个例子数据量很小，你感觉重复存储几次没啥大不了，但如果键非常多、非常长，且存在大量公共前缀（现实中确实经常有这种情况，比如证件号），那么 Trie 树就能节约大量的内存空间。

方便处理前缀操作

举个例子就明白了：

```
// Trie 树的键类型固定为 String 类型，值类型可以是泛型
TrieMap<Integer> map = new TrieMap<>();
```

```

map.put("that", 1);
map.put("the", 2);
map.put("them", 3);
map.put("apple", 4);

// "the" 是 "themxyz" 的最短前缀
System.out.println(map.shortestPrefixOf("themxyz")); // "the"

// "them" 是 "themxyz" 的最长前缀
System.out.println(map.longestPrefixOf("themxyz")); // "them"

// "tha" 是 "that" 的前缀
System.out.println(map.containsKeyWithPrefix("tha")); // true

// 没有以 "thz" 为前缀的键
System.out.println(map.containsKeyWithPrefix("thz")); // false

// "that", "the", "them" 都是 "th" 的前缀
System.out.println(map.keysWithPrefix("th")); // ["that", "the", "them"]

```

除了 `keysWithPrefix` 方法的复杂度取决于返回结果的长度，其他前缀操作的复杂度都是 $O(L)$ ，其中 L 是前缀字符串长度。

你想想上面这几个操作，用 `HashMap` 或者 `TreeMap` 能做到吗？应该只能强行遍历所有键，然后一个个比较字符串前缀，复杂度非常高。

话说，这个 `keysWithPrefix` 方法，是不是很适合做自动补全功能呢？

可以使用通配符

还是举个例子：

```

// Trie 树的键类型固定为 String 类型，值类型可以是泛型
// 支持通配符匹配，"." 可以匹配任意一个字符
TrieMap<Integer> map = new TrieMap<>();

map.put("that", 1);
map.put("the", 2);
map.put("team", 3);
map.put("zip", 4);

// 匹配 "t.a." 的键有 "team", "that"
System.out.println(map.keysWithPattern("t.a.")); // ["team", "that"]

// 匹配 ".ip" 的键有 "zip"
System.out.println(map.containsKeyWithPattern(".ip")); // true

```

```
// 没有匹配 "z.o" 的键
System.out.println(map.containsKeyWithPattern("z.o")); // false
```

通配符匹配，是不是可以做搜索引擎的关键词匹配呢？这个功能用 HashMap 或者 TreeMap 肯定是做不到的。

可以按照字典序遍历键

举例说明：

```
// Trie 树的键类型固定为 String 类型，值类型可以是泛型
TrieMap<Integer> map = new TrieMap<>();

map.put("that", 1);
map.put("the", 2);
map.put("them", 3);
map.put("zip", 4);
map.put("apple", 5);

// 按照字典序遍历键
System.out.println(map.keys()); // ["apple", "that", "the", "them", "zip"]
```

这个功能 TreeMap 也能做到，算是和 Trie 树打了个平手，但是 HashMap 就做不到了。

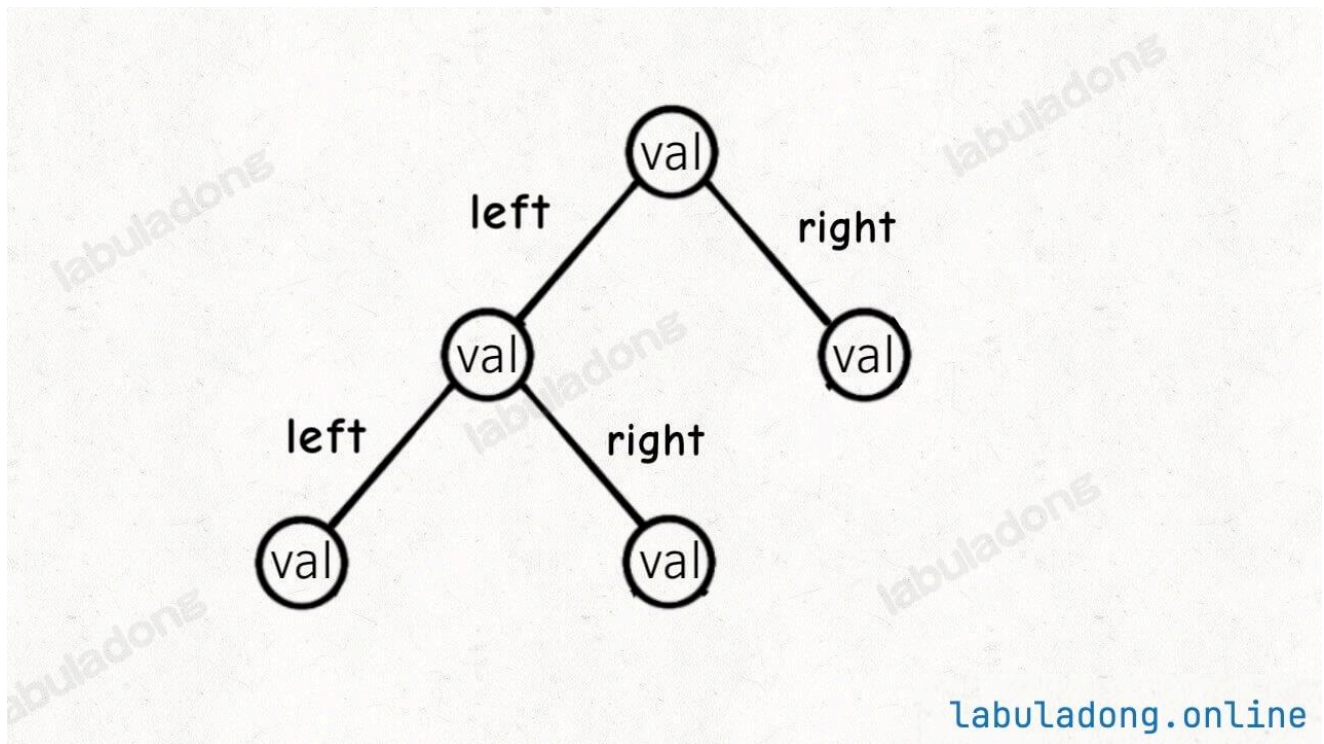
Trie 树的基本结构

Trie 树本质上就是一棵从二叉树衍生出来的多叉树。

二叉树节点的代码实现是这样：

```
// 基本的二叉树节点
class TreeNode {
public:
    int val;
    TreeNode* left;
    TreeNode* right;
};
```

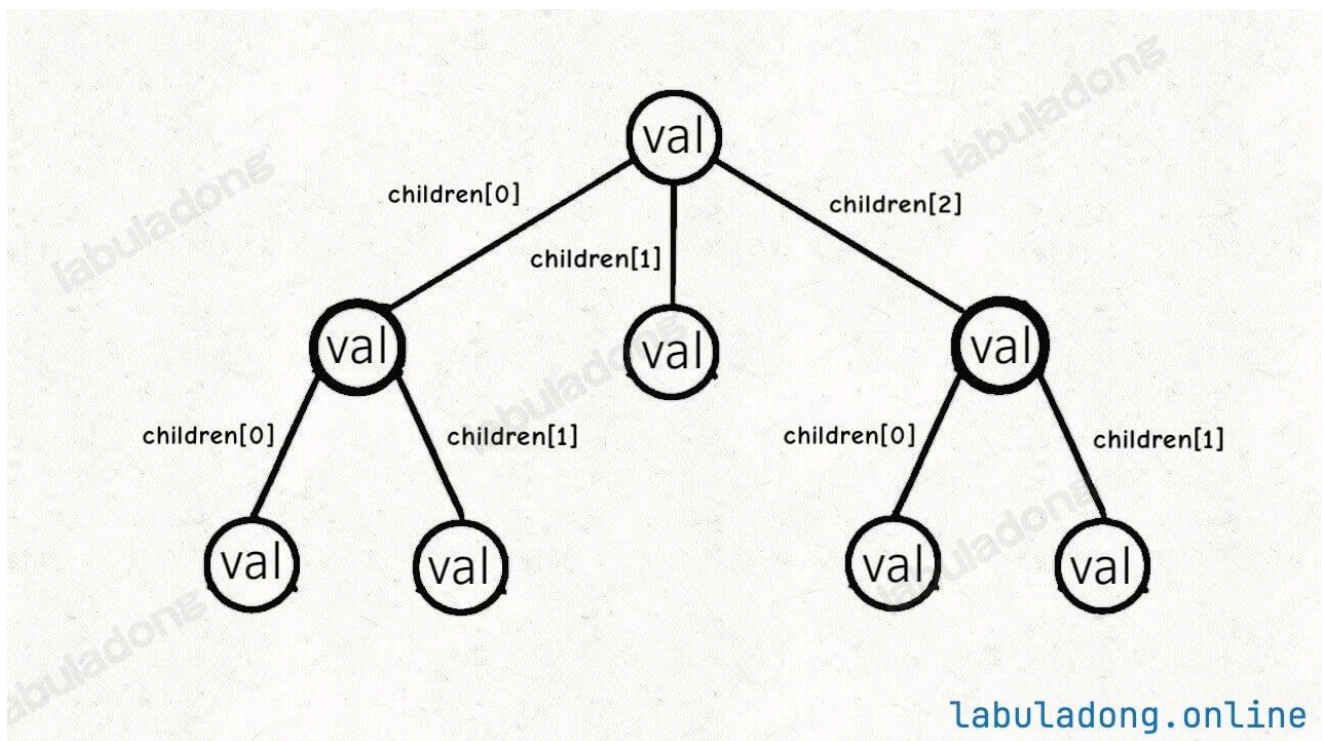
其中 left, right 存储左右子节点的指针，所以二叉树的结构是这样：



多叉树节点的代码实现是这样：

```
// 基本的多叉树节点
class TreeNode {
public:
    int val;
    vector<TreeNode*> children;
};
```

其中 `children` 数组中存储指向孩子节点的指针，所以多叉树的结构是这样：



而 `TrieMap` 中的树节点 `TrieNode` 的代码实现是这样：


```
// Trie 树节点实现
template<typename V>
struct TrieNode {
    V val = NULL;
    TrieNode<V>* children[256] = { NULL };
};
```

这个 `val` 字段存储键对应的值，`children` 数组存储指向子节点的指针。

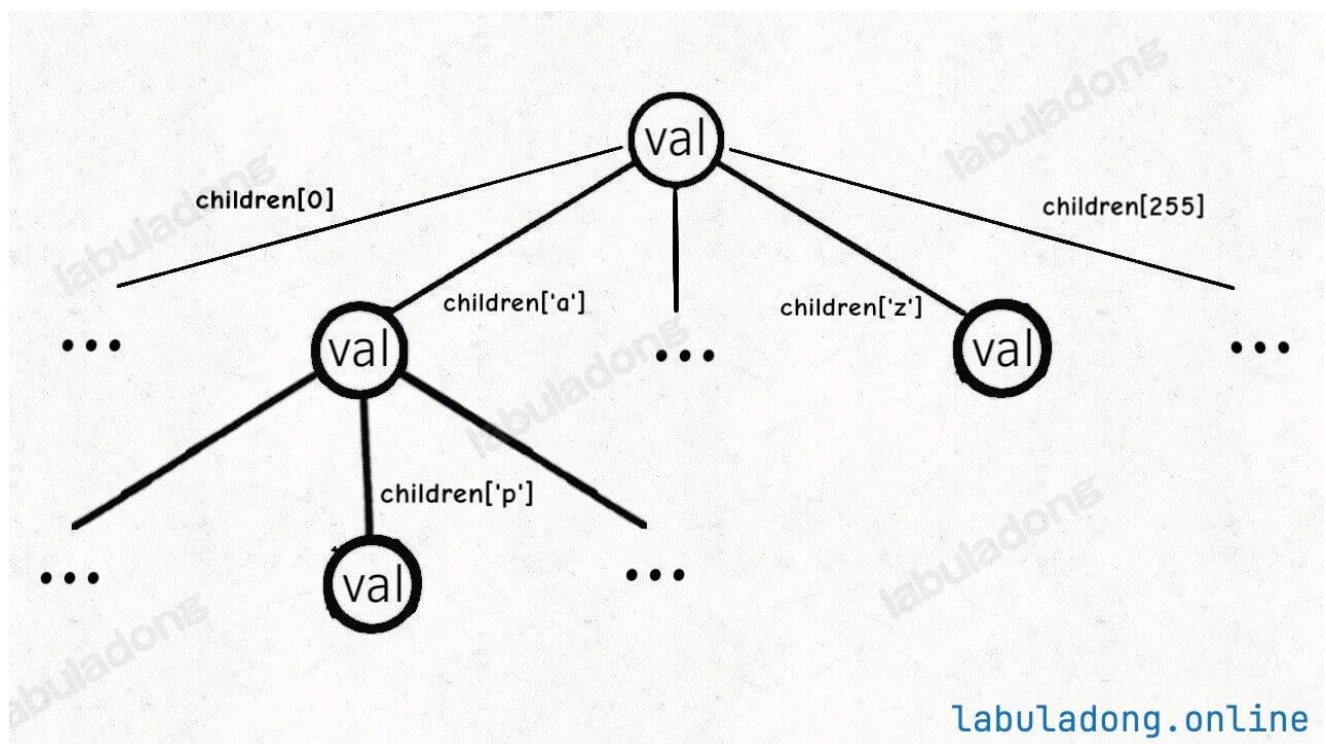
但是和之前的普通多叉树节点不同，`TrieNode` 中 `children` 数组的索引是有意义的，代表键中的一个字符。

比如说 `children[97]` 如果非空，说明这里存储了一个字符 `'a'`，因为 `'a'` 的 ASCII 码为 97。

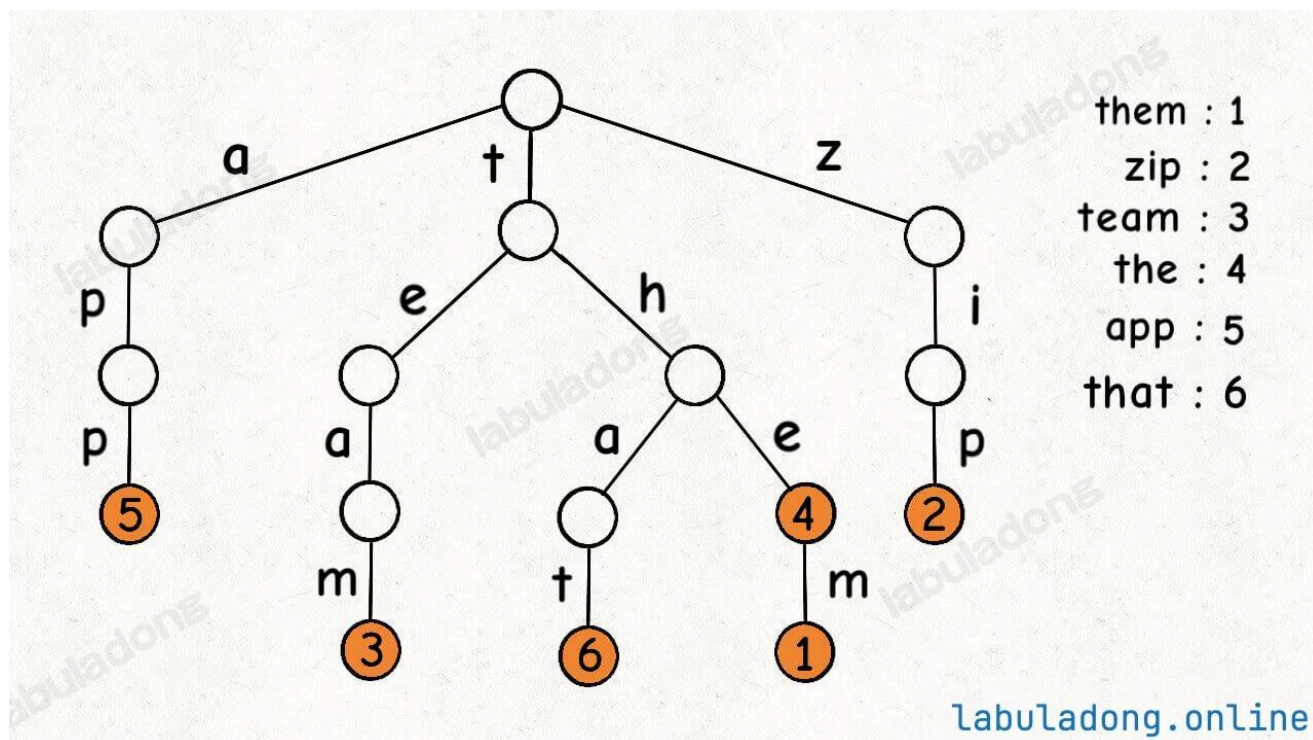
我们的模板只考虑处理 ASCII 字符，所以 `children` 数组的大小设置为 256。不过这个可以根据具体问题修改。

比如在实际做题时，题目说了只包含字符 `a-z`，那么你可以把大小改成 26；或者你不想用字符索引来映射，直接用哈希表 `HashMap<Character, TrieNode>` 也可以，都是一样的效果。

有了以上铺垫，Trie 树的结构是这样的：



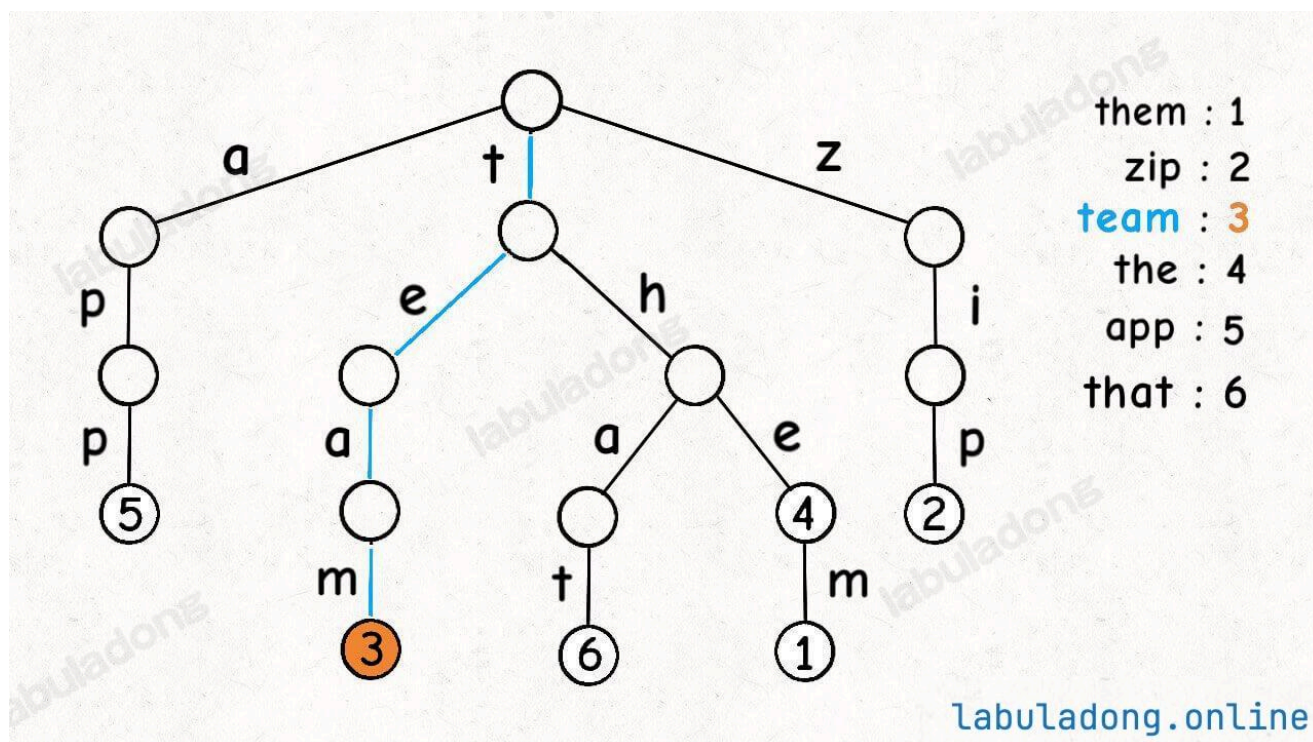
一个节点有 256 个子节点指针，但大多数时候都是空的，可以省略掉不画，所以一般你看到的 Trie 树长这样：



这是在 `TrieMap<Integer>` 中插入一些键值对后的样子，白色节点代表 `val` 字段为空，橙色节点代表 `val` 字段非空。

这里要特别注意，`TrieNode` 节点本身只存储 `val` 字段，并没有一个字段来存储字符，字符是通过子节点在父节点的 `children` 数组中的索引确定的。

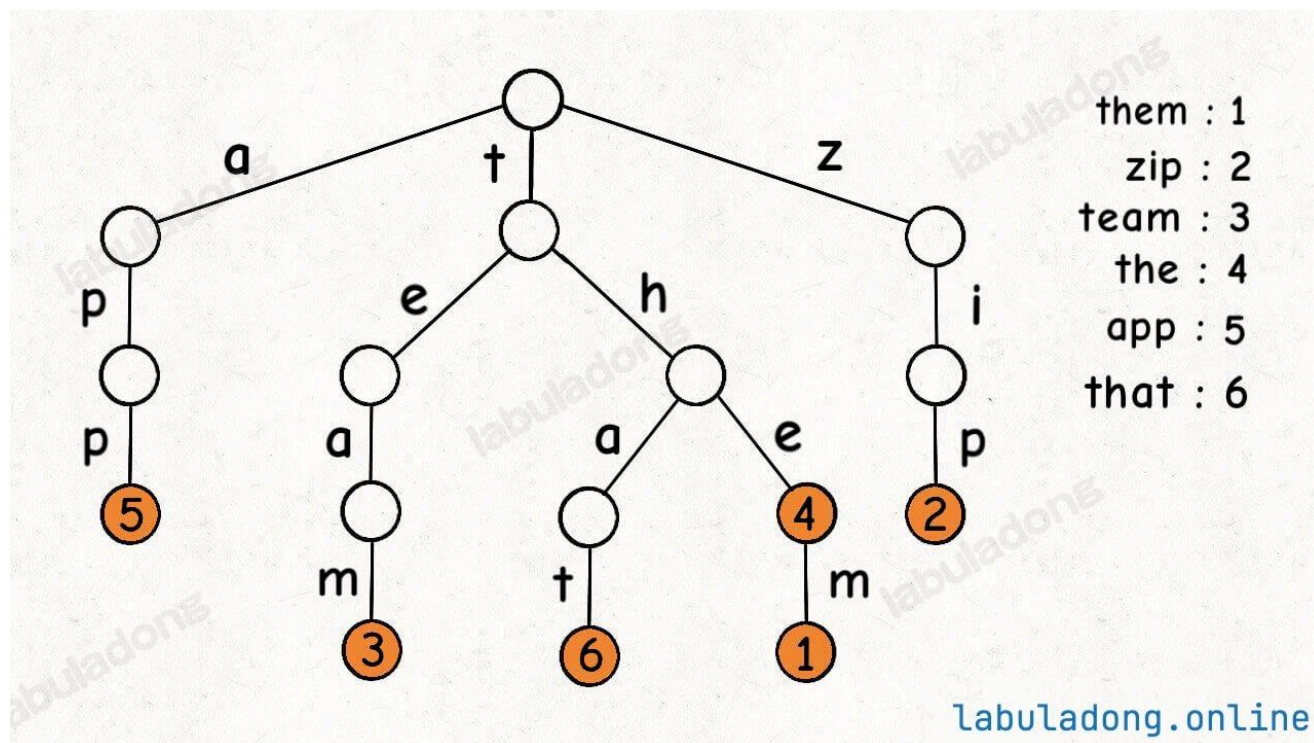
形象理解就是，Trie 树用「树枝」存储字符串（键），用「节点」存储字符串（键）对应的数据（值）。所以我在图中把字符标在树枝，键对应的值 `val` 标在节点上：



现在你应该知道为啥 Trie 树也叫前缀树了，因为其中的字符串共享前缀，相同前缀的字符串集中在 Trie 树中的一个子树上，给字符串的处理带来很大的便利。

TrieMap API

我们看一下 TrieMap 的 API，为了举例 API 的功能，假设 TrieMap 中已经存储了如下键值对：



```
template <typename V>
class TrieMap {
public:
    // 在 Map 中添加 key
    void put(string key, V val);

    // 删除键 key 以及对应的值
    void remove(string key);

    // 搜索 key 对应的值，不存在则返回 null
    // get("the") -> 4
    // get("tha") -> null
    V get(string key);

    // 判断 key 是否存在 Map 中
    // containsKey("tea") -> false
    // containsKey("team") -> true
    bool containsKey(string key);

    // 在 Map 的所有键中搜索 query 的最短前缀
    // shortestPrefixOf("themxyz") -> "the"
    string shortestPrefixOf(string query);

    // 在 Map 的所有键中搜索 query 的最长前缀
    // longestPrefixOf("themxyz") -> "them"
    string longestPrefixOf(string query);

    // 搜索所有前缀为 prefix 的键
```

```

// keysWithPrefix("th") -> ["that", "the", "them"]
vector<string> keysWithPrefix(string prefix);

// 判断是与否存在前缀为 prefix 的键
// hasKeyWithPrefix("tha") -> true
// hasKeyWithPrefix("apple") -> false
bool hasKeyWithPrefix(string prefix);

// 通配符 . 匹配任意字符，搜索所有匹配的键
// keysWithPattern("t.a.") -> ["team", "that"]
vector<string> keysWithPattern(string pattern);

// 通配符 . 匹配任意字符，判断是否存在匹配的键
// hasKeyWithPattern(".ip") -> true
// hasKeyWithPattern(".i") -> false
bool hasKeyWithPattern(string pattern);

// 返回 Map 中键值对的数量
int size();
};

```

二叉堆

二叉堆是一种能够动态排序的数据结构，是二叉树结构的延伸。

二叉堆的主要操作就两个，sink（下沉）和 swim（上浮），用以维护二叉堆的性质。

二叉堆的主要应用有两个，首先是一种很有用的数据结构优先级队列（Priority Queue），第二种是一种排序方法堆排序（Heap Sort）。

二叉堆的性质

二叉堆就是一种能够动态排序的数据结构。所谓动态排序，就是说我们可以不断往数据结构里面添加或删除元素，数据结构会自动调整元素的位置，使得我们可以有序地从数据结构中读取元素，这是一般的排序算法做不到的。

能动态排序的常用数据结构其实只有两个，一个是优先级队列（底层用二叉堆实现），另一个是二叉搜索树。二叉搜索树的用途更广泛，优先级队列能做的事情，二叉搜索树其实都能做。但优先级队列的 API 和代码实现相较于二叉搜索树更简单，所以一般能用优先级队列解决的问题，我们没必要用二叉搜索树。

二叉堆是怎么做到动态排序的呢？这就要说到二叉堆这种结构的性质了。

二叉堆的性质

你可以认为二叉堆是一种特殊的二叉树，这棵二叉树上的任意节点的值，都必须大于等于（或小于等于）其左右子树所有节点的值。如果是大于等于，我们称之为「大顶堆」，如果是小于等于，我们称之为「小顶堆」。

对于小顶堆，每个节点下方的所有节点的值都比它大，那么不难想象根节点就是整棵树上的最小值。同理，大顶堆的根节点就是整棵树上的最大值。所以二叉堆可以辅助我们快速找到最大值或最小值。

二叉堆还有个性质：一个二叉堆的左右子堆（子树）也是一个二叉堆。这个性质主要在 [堆排序算法的优化](#) 中有用到。

最常见的应用：优先级队列

```
class MyPriorityQueue {
public:
    // 在二叉堆堆顶插入一个元素，时间复杂度  $O(\log N)$ 
    //  $N$  为当前二叉堆中的元素个数
    void push(int x);

    // 返回堆顶元素，时间复杂度  $O(1)$ 
    // 该堆顶元素就是二叉堆中的最大值或最小值，取决于是最大堆还是最小堆
    int peek();

    // 删除堆顶元素，时间复杂度  $O(\log N)$ 
    int pop();

    // 返回堆中元素的个数，时间复杂度  $O(1)$ 
    int size();
};
```

不同编程语言提供的 API 名字可能不同，但其效果和底层实现是大同小异的。

当然，自动排序是有代价的，注意优先级队列 API 的时间复杂度，**增删元素的复杂度是 $O(\log N)$** ，其中 N 是当前二叉堆中的元素个数，回头在算法题里面用到这种结构的话，你得会计算总的时间复杂度。

为啥叫优先级「队列」？

有的读者可能会问，这明明就是二叉堆，为啥非要叫他优先级队列呢？

主要是因为这个数据结构的 API 和我们之前讲的 [标准队列 API](#) 很像，标准队列是先进先出的顺序，而二叉堆可以理解为一种会自动排序的队列，所以叫做优先级队列感觉也挺贴切的。

当然，你也一定要明白，虽然它的 API 像队列，但它的底层原理和二叉树有关，和队列没啥关系。

另一种应用：堆排序

这种排序算法其实不用专门去学的。它的原理特别简单，就相当于把一个乱序的数组都 `push` 到一个二叉堆（优先级队列）里面，然后再一个个 `pop` 出来，就得到了一个有序的数组：

```
// 堆排序伪码，对 arr 原地排序
// 时间复杂度 O(NlogN)，空间复杂度 O(N)
vector<int> heapSort(vector<int>& arr) {
    vector<int> res(arr.size());
    MyPriorityQueue pq;
    for (int x : arr)
        pq.push(x);
    // 元素出堆的顺序是有序的
    for (int i = 0; i < arr.size(); i++)
        res[i] = pq.pop();
    return res;
}
```

当然，正常的堆排序算法的代码并不依赖优先级队列，且空间复杂度是 $O(1)$ 。那是因为它把 push 和 pop 的代码逻辑展开了，再加上直接在数组上原地建堆，这样就不需要额外的空间了。

但堆排序的本质还是依靠二叉堆的性质来排序元素，等我手把手带你实现优先级队列之后，你就可以自己实现堆排序了。

在数组上原地建堆？

这里提到了在数组上原地建堆，细心的读者可能会有疑惑：刚才不是说二叉堆是一种特殊的二叉树吗？怎么可能不使用额外的空间复杂度，直接在数组上原地创建二叉树呢？

这个问题很好，我在 [学习数据结构和算法的框架思维](#) 中有阐述其中的原因。

二叉树是一种逻辑概念，并不是说只有 `TreeNode` 类构造出来的那个结构才是二叉树，其实数组也可以抽象成一棵树，一切分治穷举的思想都可以抽象成一棵树，递归函数的那个递归栈也可以理解成一棵树。

当然，如果你是初学者，上面的话你可能看起来有些绕。不用着急，后面的文章会让你的理解更加深入，等你真的理解这句话之后，所有的算法对你来说都小菜一碟了。

简化版优先级队列

我们实现的这个简化版优先级队列有如下限制：

- 1、不支持泛型，仅支持存储整数类型的元素。
- 2、不考虑扩容的问题，队列的容量在创建时固定，假设插入的元素数量不会超过这个容量。
- 3、底层仅实现一个小顶堆（即根节点是整个堆中的最小值），不支持自定义比较器。

基于上面这些限制，这个简化版优先级队列的 API 如下：

```
class SimpleMinPQ {
public:
    // 创建一个容量为 capacity 的优先级队列
```

```

SimpleMinPQ(int capacity);

// 返回队列中的元素个数
int size();

// 向队列中插入一个元素
void push(int x);

// 返回队列中的最小元素（堆顶元素）
int peek();

// 删除并返回队列中的最小元素（堆顶元素）
int pop();
};

// 使用方法
SimpleMinPQ pq(10);
pq.push(3);
pq.push(4);
pq.push(1);
pq.push(2);
cout << pq.pop() << endl; // 1
cout << pq.pop() << endl; // 2
cout << pq.pop() << endl; // 3
cout << pq.pop() << endl; // 4

```

二叉堆的难点在于 **你在插入或删除元素时，还要保持堆的性质。**

增：push/swim 方法插入元素

核心步骤

以小顶堆为例，向小顶堆中插入新元素遵循两个步骤：

- 1、先把新元素追加到二叉树底层的最右侧，保持完全二叉树的结构。此时该元素的父节点可能比它大，不满足小顶堆的性质。
- 2、为了恢复小顶堆的性质，需要将这个新元素不断上浮（swim），直到它的父节点比它小为止，或者到达根节点。此时整个二叉树就满足小顶堆的性质了。

删：pop/sink 方法删除元素

核心步骤

以小顶堆为例，删除小顶堆的堆顶元素遵循两个步骤：

- 1、先把堆顶元素删除，把二叉树底层的最右侧元素摘除并移动到堆顶，保持完全二叉树的结构。此时堆顶元素可能比它的子节点大，不满足小顶堆的性质。

2、为了恢复小顶堆的性质，需要将这个新的堆顶元素不断下沉（sink），直到它比它的子节点小为止，或者到达叶子节点。此时整个二叉树就满足小顶堆的性质了。

查：peek 方法查看堆顶元素

这个很简单吧，直接返回根节点的值就行了，我就不多说了。

在数组上模拟二叉树

在之前的所有内容中，我都把二叉堆作为一种二叉树来讲解。

```
class HeapNode {
    constructor(val) {
        this.val = val;
        this.left = null;
        this.right = null;
        this.parent = null;
    }
}
```

但实际上，我们在代码实现的时候，不会用类似 HeapNode 的节点类来实现，而是用数组来模拟二叉树结构。

用数组模拟二叉树的原因

第一个原因是前面介绍 [数组](#) 和 [链表](#) 时说到的，链表节点需要一个额外的指针存储相邻节点的地址，所以相对数组，链表的内存消耗会大一些。我们这里的 HeapNode 类也是链式存储的例子，和链表节点类似，需要额外的指针存储父节点和子节点的地址。

第二个原因，也是最主要的原因，是时间复杂度的问题。仔细想一下前面我给你展示的 push 和 pop 方法的操作过程，它们的第一步是什么？是不是要找到二叉树最底层的最右侧元素？

你如何拿到二叉树的底层最右侧节点？你需要层序遍历或递归遍历二叉树，时间复杂度是 $O(N)$ ，进而导致 push 和 pop 方法的时间复杂度退化到 $O(N)$ ，这显然是不可接受的。

如果用数组来模拟二叉树，就可以完美解决这个问题，在 $O(1)$ 时间内找到二叉树的底层最右侧节点。

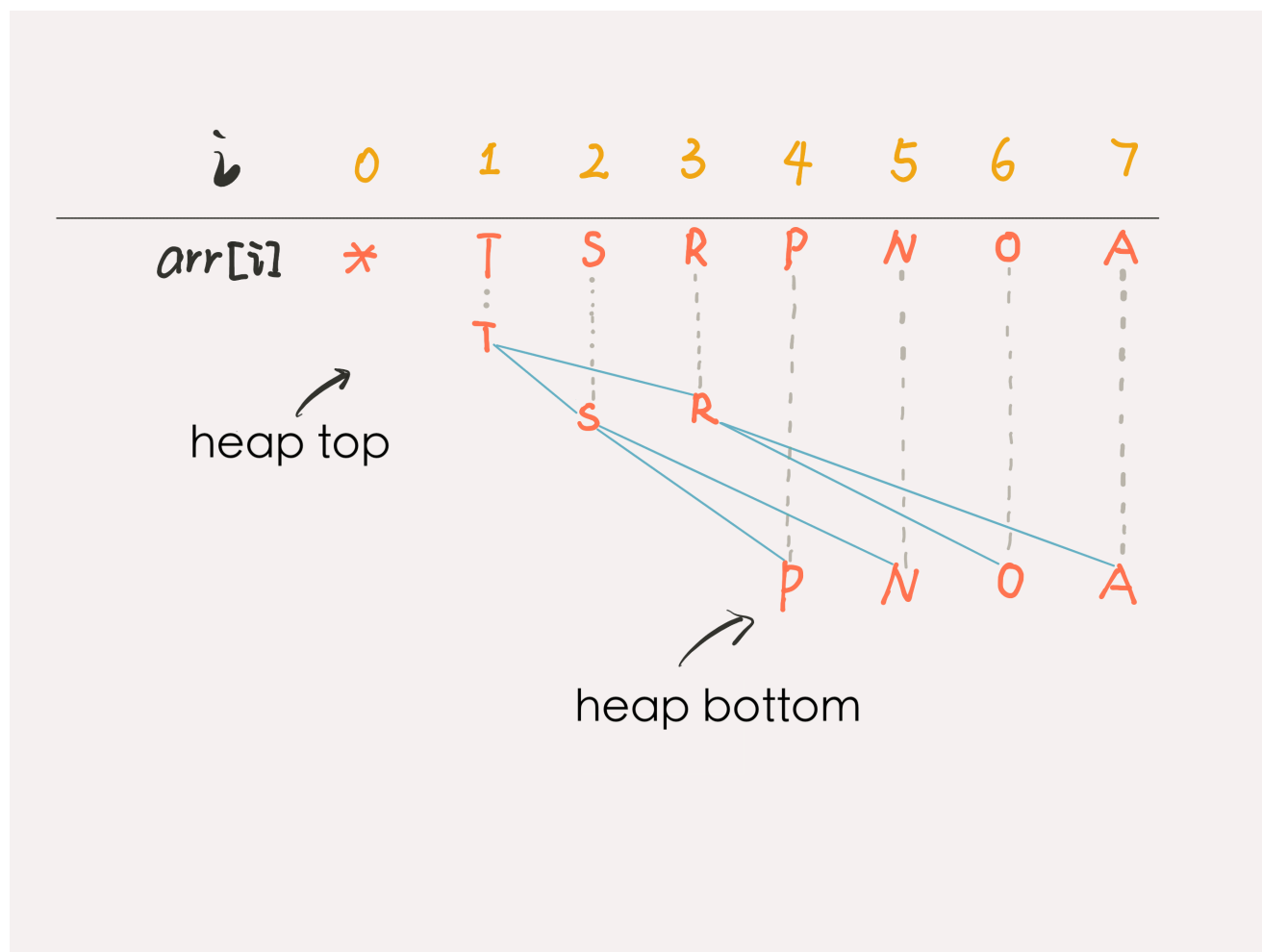
完全二叉树是关键

想要用数组模拟二叉树，前提是这个二叉树必须是**完全二叉树**。就是除了最后一层，其他层的节点都是满的，最后一层的节点都靠左排列。

由于完全二叉树上的元素都是紧凑排列的，我们可以用数组来存储。

直接在数组的末尾追加元素，就相当于在完全二叉树的最后一层从左到右依次填充元素；数组中最后一个元素，就是完全二叉树的底层最右侧的元素，完美契合我们实现二叉堆的场景。

看这幅图就明白了：

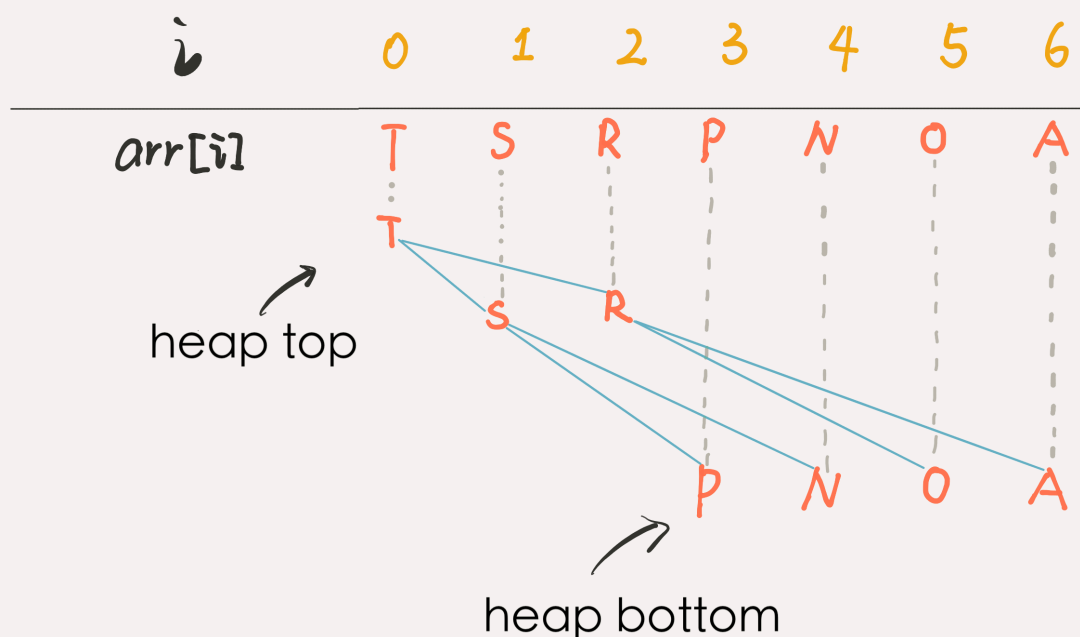


在这个数组中，索引 0 空着不用，就可以根据任意节点的索引计算出父节点或左右子节点的索引：

```
// 父节点的索引
int parent(int node) {
    return node / 2;
}
// 左子节点的索引
int left(int node) {
    return node * 2;
}
// 右子节点的索引
int right(int node) {
    return node * 2 + 1;
}
```

有读者会问，为啥数组中索引 0 要空着不用，从 1 开始存储元素呢？

其实从 0 开始也是可以的，稍微改一改计算公式就行了：



```
// 父节点的索引
int parent(int node) {
    return (node - 1) / 2;
}

// 左子节点的索引
int left(int node) {
    return node * 2 + 1;
}

// 右子节点的索引
int right(int node) {
    return node * 2 + 2;
}
```

如果你去查堆结构相关的资料和代码，可能会看到从索引 1 开始存储元素的情况更多，所以上面都提了一下，反正我感觉两种写法都差不多。

考虑到本站可视化面板中支持的二叉堆结构，以及后面讲排序算法时讲到的堆排序算法，都是以索引 0 开始存储元素的，所以下面的代码实现也是以索引 0 开始存储元素为例。

代码实现

下面是一个简化版的小顶堆优先级队列核心逻辑的实现，没有特别处理边界情况，供你参考：

```

#include <iostream>
#include <vector>
using namespace std;

class SimpleMinPQ {
    // 底层使用数组实现二叉堆
    vector<int> heap;

    // 堆中元素的数量
    int size;

    // 父节点的索引
    static int parent(int node) {
        return (node - 1) / 2;
    }

    // 左子节点的索引
    static int left(int node) {
        return node * 2 + 1;
    }

    // 右子节点的索引
    static int right(int node) {
        return node * 2 + 2;
    }

    // 上浮操作，时间复杂度是树高  $O(\log N)$ 
    void swim(int node) {
        while (node > 0 && heap[parent(node)] > heap[node]) {
            swap(heap[parent(node)], heap[node]);
            node = parent(node);
        }
    }

    // 下沉操作，时间复杂度是树高  $O(\log N)$ 
    void sink(int node) {
        while (left(node) < size || right(node) < size) {
            // 比较自己和左右子节点，看看谁最小
            int min = node;
            if (left(node) < size && heap[left(node)] < heap[min]) {
                min = left(node);
            }
            if (right(node) < size && heap[right(node)] < heap[min]) {
                min = right(node);
            }
            if (min == node) {
                break;
            }
            // 如果左右子节点中有比自己小的，就交换

```

```

        swap(heap[node], heap[min]);
        node = min;
    }
}

public:
    // 构造函数，初始化容量
    SimpleMinPQ(int capacity) {
        heap.resize(capacity);
        size = 0;
    }

    // 返回堆的元素数量
    int getSize() const {
        return size;
    }

    // 查，返回堆顶元素，时间复杂度  $O(1)$ 
    int peek() {
        return heap[0];
    }

    // 增，向堆中插入一个元素，时间复杂度  $O(\log N)$ 
    void push(int x) {
        // 把新元素追加到最后
        heap[size] = x;
        // 然后上浮到正确位置
        swim(size);
        size++;
    }

    // 删，删除堆顶元素，时间复杂度  $O(\log N)$ 
    int pop() {
        int res = heap[0];
        // 把堆底元素放到堆顶
        heap[0] = heap[size - 1];
        size--;
        // 然后下沉到正确位置
        sink(0);
        return res;
    }
};

int main() {
    SimpleMinPQ pq(5);
    pq.push(3);
    pq.push(2);
    pq.push(1);
    pq.push(5);
    pq.push(4);

```

```

        cout << pq.pop() << endl; // 1
        cout << pq.pop() << endl; // 2
        cout << pq.pop() << endl; // 3
        cout << pq.pop() << endl; // 4
        cout << pq.pop() << endl; // 5

        return 0;
    }

```

明白了这个 SimpleMinPQ 类的实现，如果你想实现一个大顶堆 SimpleMaxPQ，只需要把 swim 和 sink 方法中元素大小比较的逻辑反过来即可，这里就不再赘述了。

完善版优先级队列

基于上面的简化版优先级队列，只要加上泛型、自定义比较器、扩容等功能，就可以实现一个比较完善的优先级队列了：

```

#include <iostream>
#include <vector>
#include <functional>
#include <stdexcept>

template <typename T>
class MyPriorityQueue {
private:
    // 堆数组
    std::vector<T> heap;

    // 堆中元素的数量
    int size;

    // 元素比较器
    std::function<bool(const T&, const T&)> comparator;

    // 父节点的索引
    int parent(int node) {
        return (node - 1) / 2;
    }

    // 左子节点的索引
    int left(int node) {
        return node * 2 + 1;
    }

    // 右子节点的索引
    int right(int node) {
        return node * 2 + 2;
    }

```

```

// 交换数组的两个元素
void swap(int i, int j) {
    std::swap(heap[i], heap[j]);
}

// 调整堆的大小
void resize(int capacity) {
    if (capacity > size) {
        heap.resize(capacity);
    }
}

// 上浮操作，时间复杂度是树高  $O(\log N)$ 
void swim(int node) {
    while (node > 0 && comparator(heap[parent(node)], heap[node])) {
        swap(parent(node), node);
        node = parent(node);
    }
}

// 下沉操作，时间复杂度是树高  $O(\log N)$ 
void sink(int node) {
    while (left(node) < size || right(node) < size) {
        // 比较自己和左右子节点，看看谁最小
        int min = node;
        if (left(node) < size && comparator(heap[min],
heap[left(node)])) {
            min = left(node);
        }
        if (right(node) < size && comparator(heap[min],
heap[right(node)])) {
            min = right(node);
        }
        if (min == node) {
            break;
        }
        // 如果左右子节点中有比自己小的，就交换
        swap(node, min);
        node = min;
    }
}

public:
    // 构造函数
    MyPriorityQueue(int capacity, std::function<bool(const T&, const T&)>
comparator)
        : heap(capacity), size(0), comparator(std::move(comparator)) {}

    // 返回堆的大小

```



```

int getSize() const {
    return size;
}

// 判断堆是否为空
bool isEmpty() const {
    return size == 0;
}

// 查，返回堆顶元素，时间复杂度  $O(1)$ 
const T& peek() const {
    if (isEmpty()) {
        throw std::underflow_error("Priority queue underflow");
    }
    return heap[0];
}

// 增，向堆中插入一个元素，时间复杂度  $O(\log N)$ 
void push(const T& x) {
    // 扩容
    if (size == heap.size()) {
        resize(2 * heap.size());
    }
    // 把新元素追加到最后
    heap[size] = x;
    // 然后上浮到正确位置
    swim(size);
    size++;
}

// 删，删除堆顶元素，时间复杂度  $O(\log N)$ 
T pop() {
    if (isEmpty()) {
        throw std::underflow_error("Priority queue underflow");
    }
    T res = heap[0];
    // 把堆底元素放到堆顶
    swap(0, size - 1);
    size--;
    // 然后下沉到正确位置
    sink(0);
    // 缩容
    if (size > 0 && size == heap.size() / 4) {
        resize(heap.size() / 2);
    }
    return res;
}
};

// 测试代码

```

```
int main() {
    // 使用lambda表达式来传递比较器
    // 小顶堆
    MyPriorityQueue<int> pq(3, [](const int& a, const int& b) { return a >
b; });
    pq.push(3);
    pq.push(1);
    pq.push(4);
    pq.push(1);
    pq.push(5);
    pq.push(9);

    // 1 1 3 4 5 9
    while (!pq.isEmpty()) {
        std::cout << pq.pop() << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

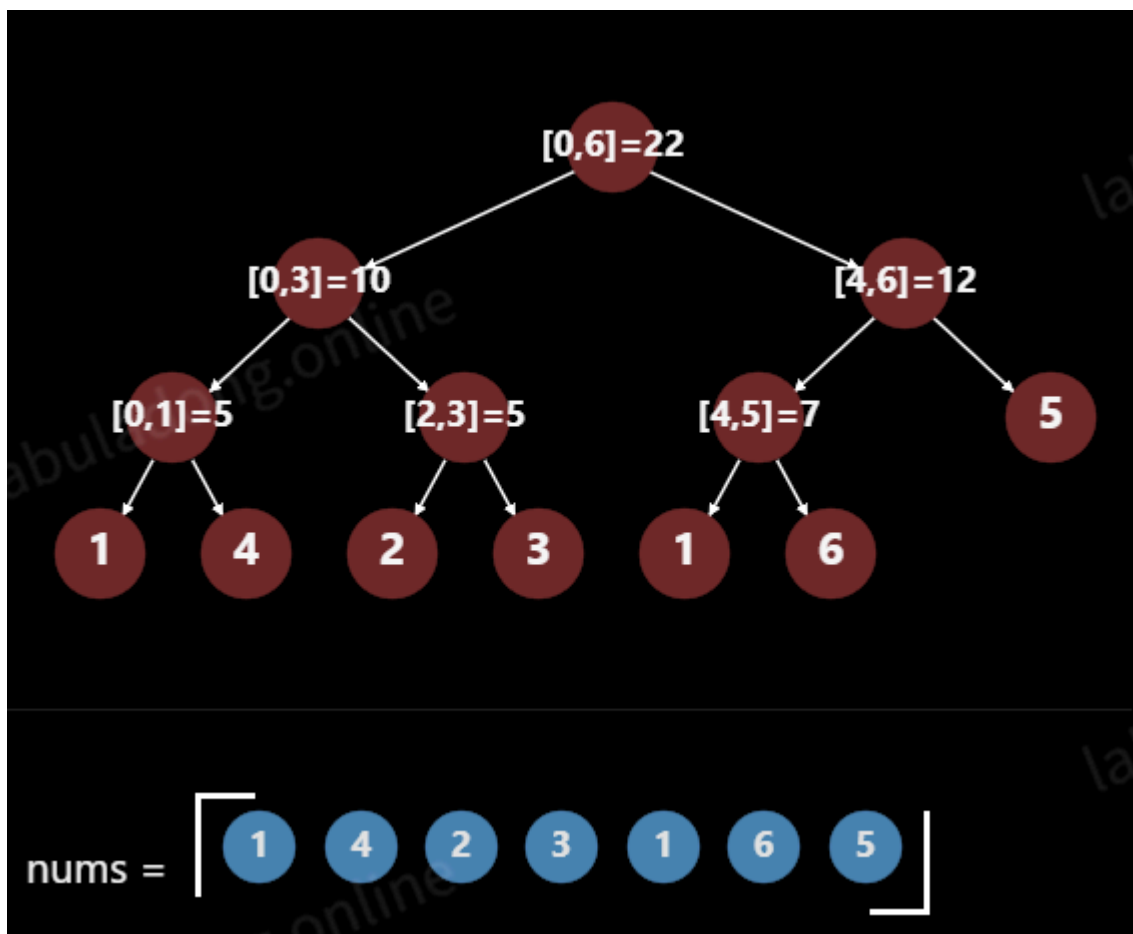
线段树核心原理

简介

线段树是 二叉树结构 的衍生，用于高效解决数组的区间查询和区间动态修改问题。

线段树可以在 $O(\log N)$ 的时间复杂度查询**任意长度**的区间元素聚合值，在 $O(\log N)$ 的时间复杂度对**任意长度**的区间元素进行动态修改，其中 N 为数组中的元素个数。

首先，基本的线段树 包含区间查询 `query` 和**单点修改** `update` 方法，可以看到这棵二叉树的叶子节点是数组中的元素，非叶子节点就是索引区间（线段）的汇总信息，也就是「线段树」这个名字的由来。



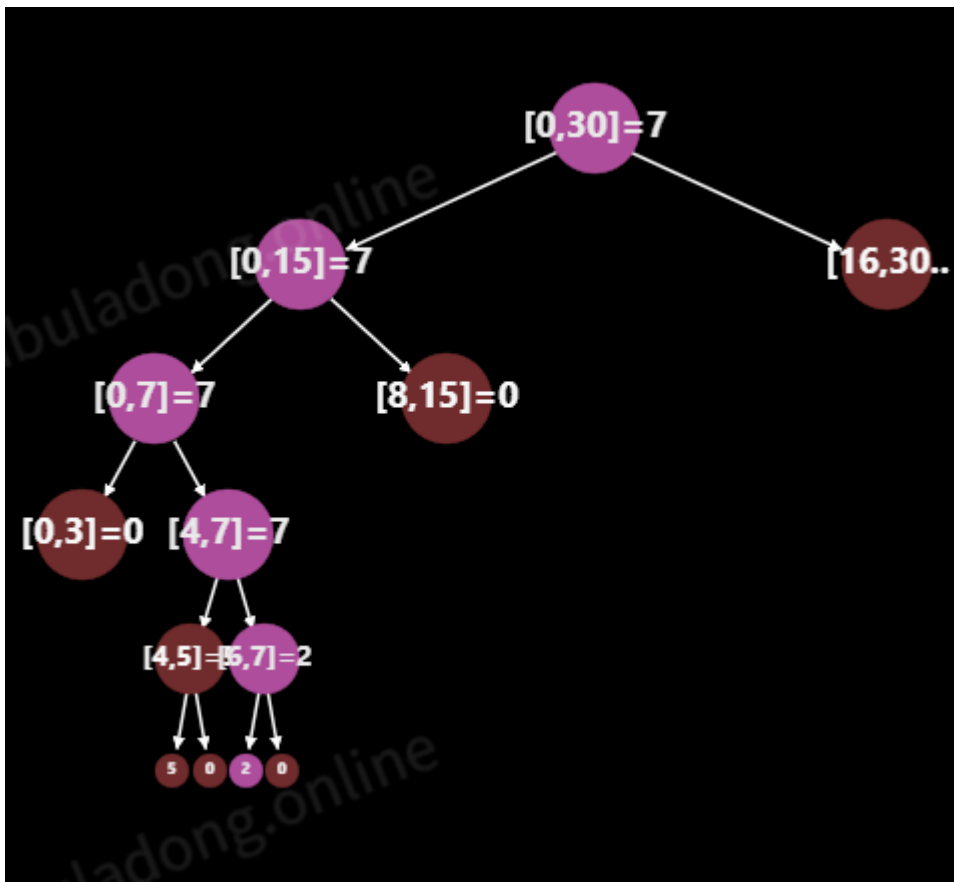
但上面这个线段树有个问题，就是必须输入 `nums` 数组进行构建，如果我们想在一个非常长的区间上进行区间操作，比如 `[0, 109]`，那么上来就需要 `109` 的空间复杂度构建线段树，这是非常浪费的。

[动态线段树的实现](#) 运用「动态开点」技巧优化线段树处理稀疏数据的内存开销。

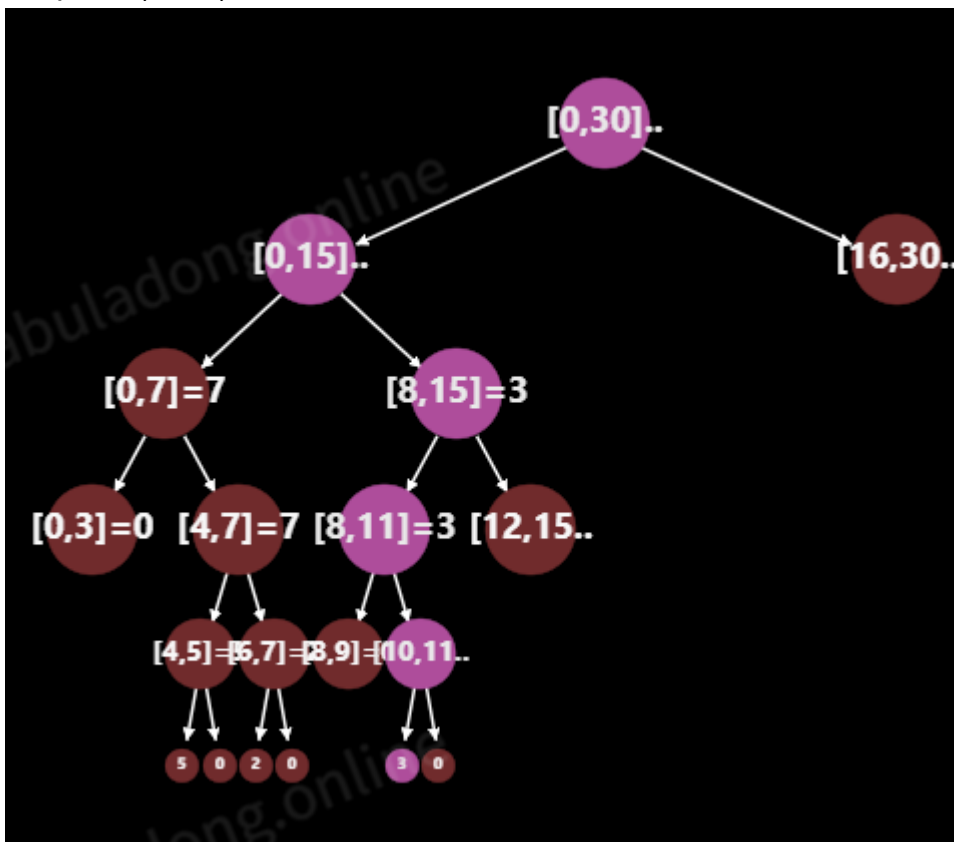
// 更新索引 6 的元素为 2

// 动态创建了相关的节点

`st.update(6, 2)`



// 更新索引 10 的元素为 3
 // 动态创建了相关的节点
 st.update(10, 3)



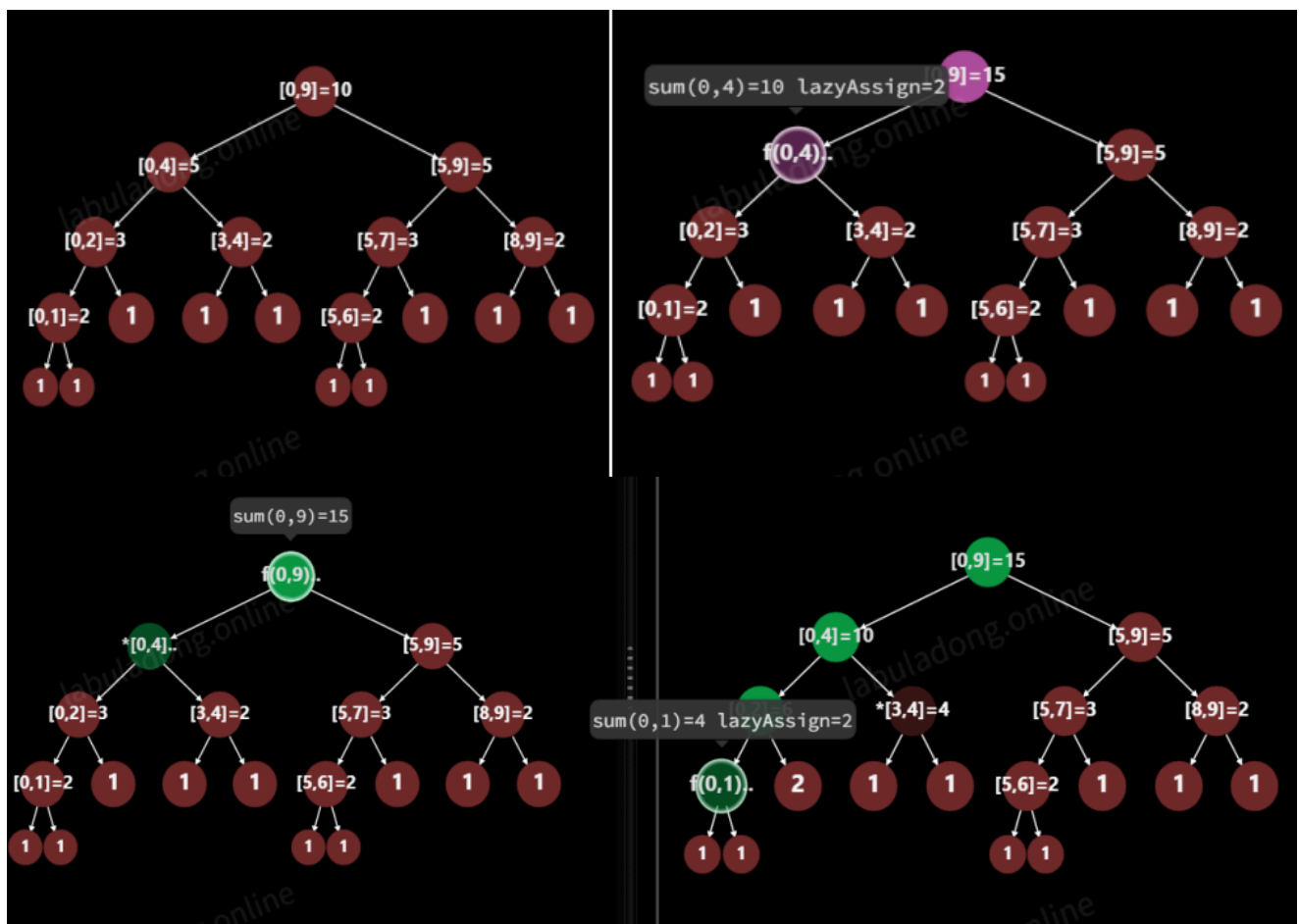
上面的实现都只支持「单点更新」，但更通用的需求是区间更新，比如把索引区间 $[i, j]$ 的元素都更新为 val 。[懒更新线段树的实现](#) 运用「懒更新」技巧，给线段树新增 `rangeAdd/rangeUpdate` 方法，可以在 $O(\log N)$ 时间复杂度内完成任意长度的区间更新。

`rangeUpdate` 方法更新区间时，并不需要立即更新区间内的所有叶子节点，而是将更新的值缓存在某个非叶子节点中，当调用 `query` 方法进行区间查询时，才逐渐将更新的值向叶子节点传播：

```
let sumTree = LazySegmentTree.createSumByArray(  
  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]  
);
```

```
// 更新区间 [0, 4] 的所有元素为 2  
// 注意我们仅仅更新了 [0, 4] 这个节点，  
// 并没有立刻更新所有叶子节点  
sumTree.rangeUpdate(0, 4, 2);  
// [2, 2, 2, 2, 2, 1, 1, 1, 1, 1]  
  
// 尝试查询区间 [0, 4] 的元素和  
// 命中了带有懒标记的 [0, 4] 节点，直接返回  
console.log(sumTree.query(0, 4));  
// 2+2+2+2+2 = 10
```

```
// 尝试查询区间 [0, 1] 的元素和  
// 此时需要查询 [0, 4] 节点的子节点  
// 所以将懒标记下推到节点 [0, 1] 和节点 [3, 4]  
console.log(sumTree.query(0, 1));  
// 2+2 = 4
```



使用场景

在 [选择排序](#) 中，我们会尝试解决一个需求，就是计算 `nums` 数组中从索引 `i` 开始到末尾的最小值。

我们将提出一种使用 `suffixMin` 数组的优化尝试，即提前预计算一个 `suffixMin` 数组，使得 `suffixMin[i] = min(nums[i..])`，这样就可以在 $O(1)$ 时间内查询 `nums[i..]` 的最小值：

```
int[] nums = new int[]{3, 1, 4, 2};
// suffixMin[i] 表示 nums[i..] 中的最小值
int[] suffixMin = new int[nums.length];

// 从后往前计算 suffixMin
suffixMin[nums.length - 1] = nums[nums.length - 1];
for (int i = nums.length - 2; i >= 0; i--) {
    suffixMin[i] = Math.min(nums[i], suffixMin[i + 1]);
}

// [1, 1, 2, 2]
System.out.println(suffixMin);

// 有了 suffixMin 数组，可以在  $O(1)$  时间内查询任意 nums[0..i] 后缀的最小值

// 查询 nums[1..] 的最小值
System.out.println(suffixMin[1]); // 1
```

其实这种预计算的思路是非常常见的，不仅是求最小值，求和、求最大值、求乘积等场景，都可以通过预计算数组来优化查询效率。

后面的章节你就会学到，这种技巧可以统一归为 [前缀和技巧](#)，主要用于处理区间查询问题。

但这个技巧有它的局限性，即 `nums` 数组本身不能变化。一旦 `nums[i]` 变化了，那么 `suffixMin[0..i]` 的值都会失效，需要 $O(N)$ 的时间复杂度重新计算 `suffixMin` 数组。

对于这种希望对整个区间进行查询，同时支持动态修改元素的场景，是线段树结构的应用场景。

线段树的核心 API

线段树结构可以有多种变体及复杂的优化，我们这里先聚焦最核心的两个 API：

```
class SegmentTree {
    // 构造函数，给定一个数组，初始化线段树，时间复杂度  $O(N)$ 
    // merge 是一个函数，用于定义 query 方法的行为
    // 通过修改这个函数，可以让 query 函数返回区间的元素和、最大值、最小值等
    public SegmentTree(int[] nums, Function<Integer, Integer> merge) {}
```

```
// 查询闭区间 [i, j] 的元素和（也可能是最大最小值，取决于 merge 函数），时间复杂度 O(logN)
public int query(int i, int j) {}

// 更新 nums[i] = val，时间复杂度 O(logN)
public void update(int i, int val) {}
}
```

类比之前说的 `suffixMin` 数组，假设 `nums` 数组的元素个数为 N ：

- `suffixMin[i]` 可以在 $O(1)$ 时间内查询 `nums[i..]` 后缀的最小值；线段树的 `query` 方法不仅可以查询后缀，还可以查询任意 $[i, j]$ 区间，时间复杂度均为 $O(\log N)$ 。
- 当底层 `nums` 数组中的任意元素变化时，需要重新计算 `suffixMin` 数组，时间复杂度为 $O(N)$ ；而线段树的 `update` 方法可以在 $O(\log N)$ 时间内完成元素的修改。

线段树不仅仅支持计算区间的最小值，只要修改 `merge` 函数，就可以支持计算区间元素和、最大值、乘积等。

线段树的核心原理

树高为什么是 $O(\log N)$ ？

在代码实现部分你会看到，最简单的线段树构造方法，是递归地将 `nums` 数组从**正中间**切分，然后递归地将左右子数组构造成线段树，直到区间长度为 1。

由于每次递归构造线段树都是从正中间切分的，左右子树的元素个数基本相同，所以整棵二叉树是平衡的，即树高是 $O(\log N)$ ，其中 N 是 `nums` 数组的元素个数。

query 为什么是 $O(\log N)$

树的高度为 $O(\log N)$ ，`query` 方法的本质是从根节点开始遍历这棵二叉树，需要遍历的节点个数是 $\log N$ 的常数倍，所以 `query` 方法的时间复杂度是 $O(\log N)$ 。

如果恰好有一个非叶子节点记录着区间 $[i, j]$ 的元素和，那么就可以直接返回；如果没有， $[i, j]$ 区间会被分裂成两个子区间，递归查询左右子区间的元素和，然后合并结果。

update 为什么是 $O(\log N)$

数组的元素就是叶子节点，`update` 方法的本质是修改叶子节点的值，顺带会更新从根节点到该叶子节点的路径上的所有非叶子节点的值。因为树高为 $O(\log N)$ ，所以 `update` 方法的时间复杂度也是 $O(\log N)$ 。

上面就是线段树结构的核心原理，具体的实现及优化将安排在后面的数据结构设计章节 [基本线段树的代码实现](#)，那时候你应该可以熟练地操作和构造二叉树了，所以线段树的代码会比较容易理解。

数据压缩和霍夫曼树

霍夫曼树是二叉树结构的经典应用，它是一种最优前缀编码树，常用于数据压缩。

本文会介绍霍夫曼编码的原理，并对比几种常见的数据压缩思路。
具体的代码实现会放到数据结构设计章节中，带你基于霍夫曼编码实现一个压缩程序。

浅谈数据压缩

我们可以把数据压缩算法分为两大类，分别是 **无损压缩** 和 **有损压缩**。

无损压缩 是指压缩后的数据可以完全还原，不会丢失任何信息。

比如我们把若干文件打包成一个 zip 压缩包，这个 zip 所需的磁盘空间会更小，且解压后可以完全还原原始文件，这就叫无损压缩。

有损压缩 是指压缩后的数据会丢失一些信息，但是压缩率更高（即压缩后的数据占用空间更小）。

比如我们经常会有图片压缩的需求，有些图像处理工具能够在不明显影响图片质量的前提下，显著降低图片的磁盘占用空间，这就是有损压缩。

那么我们来思考几个问题。

- 1、有损压缩是怎么做到在丢失信息的前提下，依然保证图片质量的呢？
- 2、有损压缩会丢失一些信息，换取空间的减少，这个可以理解。但无损压缩是怎么做到不丢失任何信息的前提下，压缩数据的呢？

首先，有损压缩肯定是会降低图片质量的，只不过这个降低的程度在我们的可接受范围内。

还是以图片压缩为例，人眼对「亮度」比较敏感，对「色度」不敏感。那么我们就可以用低精度的数据类型来表示「色度」，这样即便丢失了一些「色度」的信息，也几乎看不出区别。

但无损压缩不能这么搞，因为它要保证压缩后的数据可以再完全还原，所以**无损压缩的本质就是编码和解码**。

举个简单的例子，hahahahahahaha 这个字符串，我可以编码为 $ha \times 7$ ，解码时只需要把 ha 重复 7 次，就可以还原出原始字符串。

无损压缩的效果取决于压缩算法是否能够充分挖掘出原始数据中的冗余信息。

越是通用的压缩算法，能够挖掘出的冗余信息就越少，压缩率就越低；越是专用的压缩算法，能够挖掘出的冗余信息就越多，压缩率就越高。

比方说音频文件包含声波信息，专业的音频压缩算法能够找到声波信息中的冗余进行压缩，效果就好；而通用的 zip 压缩算法将音频文件也视为普通的字节流，无法挖掘声波信息的冗余，压缩效果自然就差。

所以没有全能的压缩算法，只能在通用性、压缩率和性能之间进行权衡。

本文讲解的霍夫曼编码，就是一种通用无损压缩算法，将原始数据输入霍夫曼算法，会得到压缩后的数据和一个码表，解码时需要根据码表还原出原始数据。

定长编码 vs 变长编码

既然说到了编码和解码，就不得不聊一聊定长编码（fixed-length encoding）和变长编码（variable-length encoding）。

ASCII 编码是一种定长编码，它将每个字符编码为 8 位二进制数，即一个字节。

UTF-8 编码是一种变长编码，它将每个字符编码为 1 到 4 个字节。

定长编码最大的优势是可以「随机访问」，因为每个字符的编码长度都是固定的，所以可以很容易地根据索引计算出字符的位置。

变长编码的优势是存储效率高，比如 UTF-8 编码就是一种变长编码，存储英文字符时占用一个字节，存储中文时占用三个字节，通用性和存储效率都比 ASCII 编码强。但由于每个字符的编码长度不固定，无法通过索引进行「随机访问」。

到这里不妨思考一下，现代编辑器基本都是用 UTF-8 编码了，而对字符串进行随机访问又是编辑器的基本功能，如果每次访问都要线性扫描，显然效率很差，那么编辑器是怎么解决这个问题的呢？

回到数据压缩的场景，比方说 `aaabacc` 这个字符串有 7 个小写字母，用 ASCII 编码需要 $7 \times 8 = 56$ bit，我现在想对它进一步压缩，应该怎么做？

因为现在只有 `a, b, c` 三种字符，所以其实不需要用 8 比特来表示每个字符，用 2 比特就够了。

比方说这样编码：

- `a` 编码为 `00`
- `b` 编码为 `01`
- `c` 编码为 `10`

那么 `aaabacc` 就可以编码为二进制的 `00000001001010`，长度为 14 比特。

这种编码方式就叫 **定长编码**，因为每个字符的编码长度都固定为 2 比特。

定长编码的好处是简单，只需要知道所有可能的字符种类，就可以确定编码了。不过定长编码的压缩效果并不是很好，因为没有充分利用字符出现的频率信息。

比方说还以 `aaabacc` 为例，既然字符 `a` 出现的频率较高，`b, c` 出现的频率低，那么能否使用更短的编码来表示 `a`，让 `b, c` 使用更长的编码呢？

其实是可以的，这就叫 **变长编码**。比如这样编码：

- `a` 编码为 `0`
- `b` 编码为 `10`
- `c` 编码为 `11`

那么 `aaabacc` 就可以编码为二进制的 `0001001111`，长度为 11 比特，比定长编码的压缩效果更好。

变长编码的难点

两个难点

- 1、如何设计编码才能保证解码的唯一性？
- 2、如何保证压缩率（编码数据尽可能短）？
- 3、如何保证解码的效率？

我们仔细分析一下上面 `aaabacc` 的例子。

这种编码方案是没有歧义的：

- a 编码为 `0`
- b 编码为 `10`
- c 编码为 `11`

如果把 a 编码为 `1`，那么编码方案中 a 和 c 的编码存在歧义：

- a 编码为 `1`
- b 编码为 `10`
- c 编码为 `11`

`aaabacc` 会被编码为二进制的 `11111011111`，但是 a 和 c 的编码存在歧义，`11` 既可以解码为 c，也可以解码为 `aa`，所以无法解码出原始数据。

对比上面两个例子可以发现一些规律：**任意一个编码，都不能是另一个编码的前缀。**

比如第二个例子中，a 编码为 `1`，而 b, c 编码的前缀都是 `1`，所以编码方案有歧义。

那么可能有读者会反驳，比如下面这个编码方案：

- a 编码为 `1`
- b 编码为 `10`
- c 编码为 `100`

虽然出现了相同的前缀，但是我们可以在解码时添加额外的比较逻辑：

读取到 `1` 时，继续向后探测两位，看看是否可能凑出 `10` 或 `100`，进而决定如何解码。

这样确实可以保证解码唯一性，**但是压缩率低，且解码效率差**。向后匹配的逻辑相当于嵌套 for 循环：

```
for (int i = 0; i < N; i++) {  
    for (int j = 1; j <= K; j++) {
```

```
    ...  
    }  
}
```

假设最长的编码长度为 K ，编码后的数据长度为 N ，那么解码的时间复杂度 $O(NK)$ 。

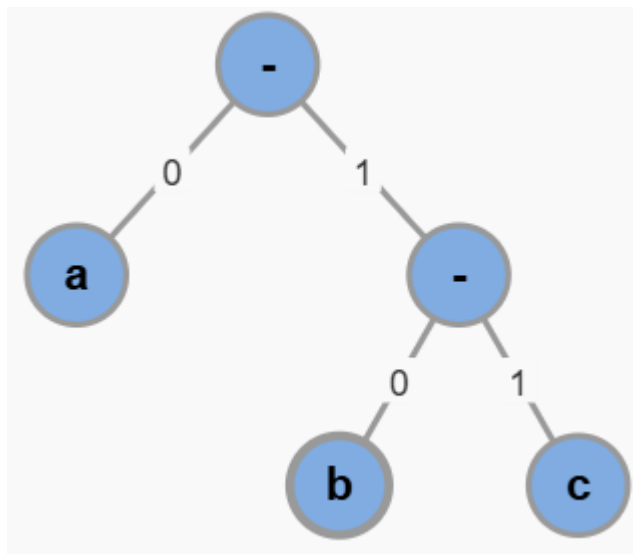
如果你能保证任意一个编码都不是另一个编码的前缀，那么就不需要尝试向后匹配了。这样一来，解码的复杂度就降到了 $O(N)$ 。

在实际的编解码场景中 N 往往很大，即便 K 较小，解码速度慢几倍也是非常糟糕的，所以我们需要保证编解码算法的时间复杂度为 $O(N)$ ，且压缩率尽可能高。

霍夫曼编码原理

霍夫曼编码就是一种变长编码方案，它借助二叉树结构构造变长编码，能够确保解码的唯一性，同时兼顾压缩效果和性能。

首先让我们来研究这样一棵二叉树：



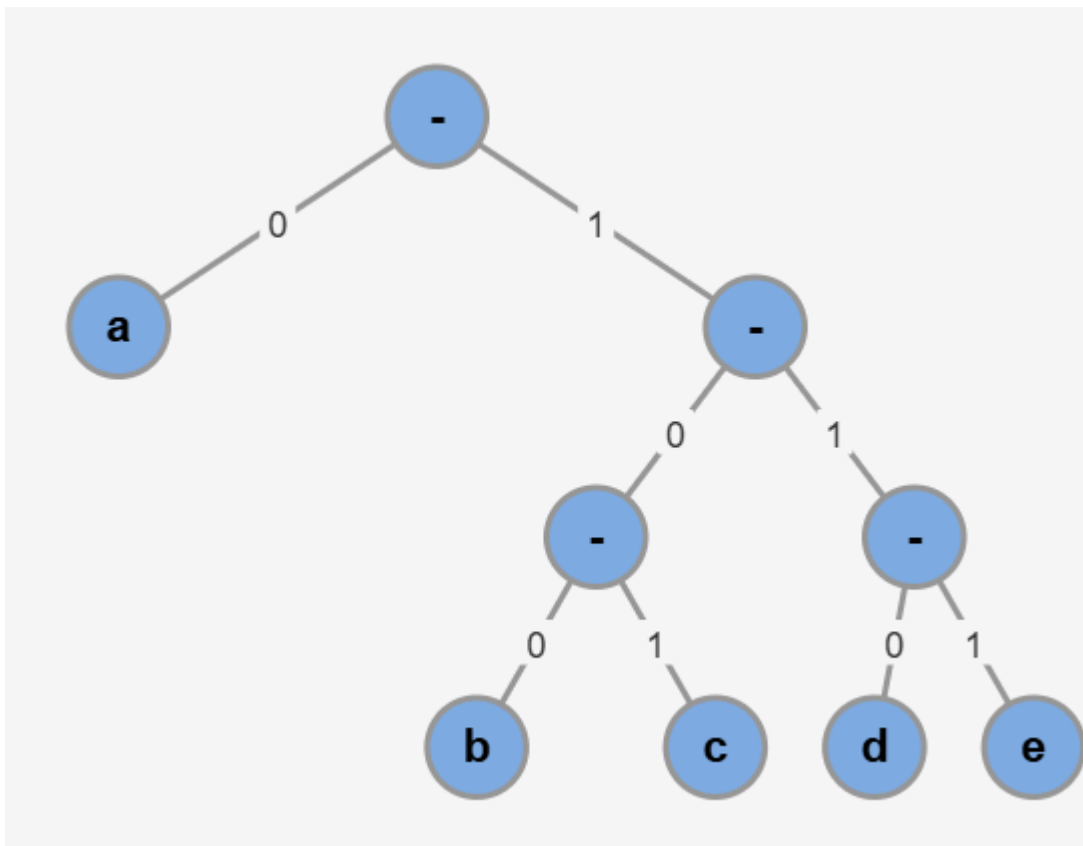
在上面这棵二叉树中，每个二叉树节点的左侧树枝代表 0 ，右侧树枝代表 1 ，叶子节点就是每个待编码的字符，从根节点到叶子节点经过的 $0, 1$ 序列就是该字符的编码：

```
a -> 0  
b -> 10  
c -> 11
```

由于每个字符都处于叶子节点，所以任意一个编码都不可能是另一个编码的前缀，必然不存在解码歧义，且解码时也不需要多余的探测逻辑，时间复杂度为 $O(N)$ 。

唯一剩下的问题就是，如何才能保证压缩率尽可能高？

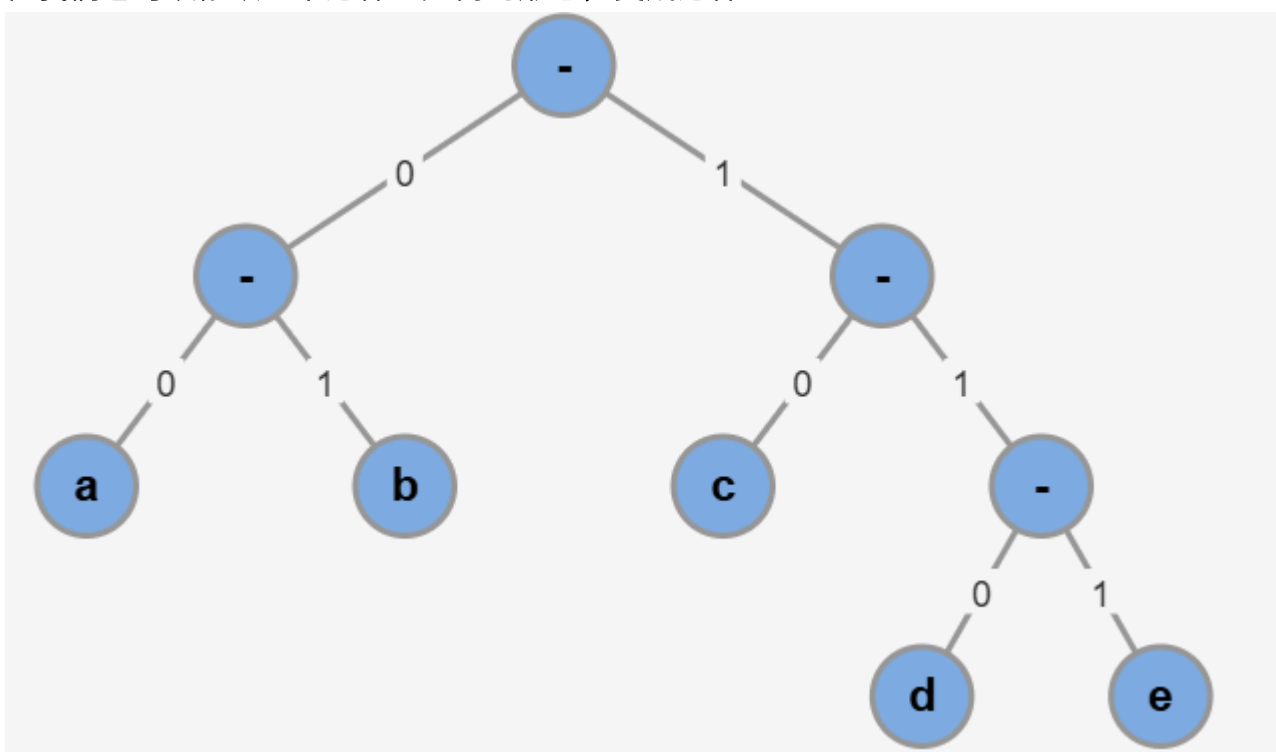
来看一个稍微复杂的场景，假设一个字符串包含 a, b, c, d, e 五种字符，它们可以构造出下面这棵二叉树：



每个字符的编码如下：

a -> 0
b -> 100
c -> 101
d -> 110
e -> 111

但我们也可以修改一下这棵二叉树的形态，变成这样：

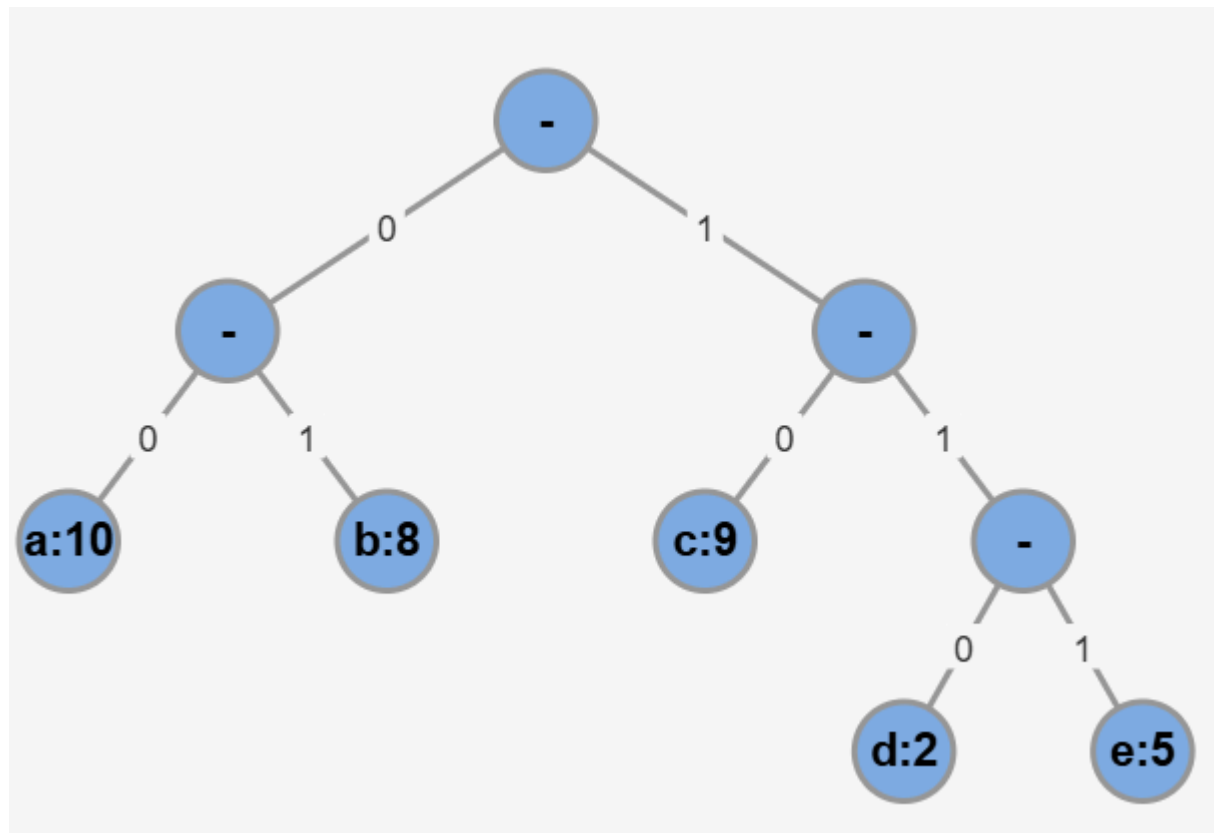


对应的编码方案是：

```
a -> 00
b -> 01
c -> 10
d -> 110
e -> 111
```

显然，到底哪种编码方案的压缩率最好，取决于每个字符出现的频率。

我们可以把每个字符出现的频率也存储到二叉树的叶子节点，假设是这样：



那么编码后的数据长度应该就是：叶子节点的值乘以从根节点到该叶子节点的路径长度，最后对所有叶子节点求和。这就是 **树的带权路径长度 (Weighted Path Length)**，简称 **WPL**。

上面的示例中，树的带权路径长度为：

$$10 * 2 + 8 * 2 + 9 * 2 + 2 * 3 + 5 * 3 = 75$$

我们想让编码后的数据长度尽可能短，其实就是寻找一棵 WPL 最小的二叉树，这棵树就叫做最优前缀编码树或霍夫曼树 (Huffman Tree)。基于这棵树计算出的编码方案，就是霍夫曼编码 (Huffman Coding)。

构造霍夫曼树的思路也不难想到，关键就是要把出现频率高的字符放在离根节点较近的地方，把出现频率低的字符放在离根节点较远的地方。

具体的代码实现会更新到数据结构设计章节的 [实现霍夫曼压缩算法](#) 具体介绍，这里暂不展开。

后续更新二叉树扩展延伸与经典数据结构设计